Fortivex Project

Pannonhalmi Főapátság Szegedi SOB Technikuma

Szakképesítés:

Szoftverfejlesztő- és tesztelő - (506131203)

Témavezető:
Rédai Dávid

A projektet készítette:
Medvecz Martin
Vigh Gréta
Halász-Szabó Dániel

Szeged, 2026.

Tartalomjegyzék

1. A projekt áttekintése 4
2. A projekt célja 4
3. SWOT-elemzés 5
3.1. Strengths (Erősségek) 5
3.2. Weaknesses (Gyengeségek) 5
3.3. Opportunities (Lehetőségek) 6
3.4. Threats (Veszélyek) 6
3.5. Fejlesztési modell 6
4. Játékmechanika és entitások 8
4.1.Ellenségtípusok és jellemzőik 8
4.2. Toronyrendszer és stratégiai lehetőségek 13
4.3. Játékmenet a gyakorlatban 19
5. Backend és fejlődési rendszer 27
5.1. Szintezési rendszer: Tapasztalati pontok és szintek kezelése 27
5.2. Teljesítmény rendszer: Teljesítmények és mérföldkövek 28
6. Rendszerarchitektúra és alkalmazott technológiák 29
6.1. Multiplatform megvalósítás: PC (Desktop) és Mobil (Android) 31
6.2.Adatbázis-felépítés és adatok perzisztenciája 31
6.3. Frontend: Webes adminisztrációs felület és főoldal 33
6.4. Frontend: Weboldal, egy felhasználó szemével 34
7.Telepítés és deployment folyamatok 38
7.1. Játék telepítése 38
7.2. Windows telepítés 38
7.3. Linux telepítés 39
7.4. Mobil telepítés 39
8. Munkamegosztás és projektmenedzsment 40
8.1. Feladatkörök a csapaton belül 41
8.2. Munkafolyamat nyomon követése (Trello/GitHub) 42
8.3. Időbeosztás és mérföldkövek 44

9. Tesztelés 48
9.1. Frontend tesztelés 48
9.2. Backend tesztelés 50
9.3. Tesztelések összegzése 52
10. Összegzés és jövőbeli fejlesztési irányok 53
11. Felhasznált források 54

1. A projekt áttekintése

A Fortivex egy multiplatformos, egyjátékos, fantasy környezetbe helyezett Tower Defense játék, ahol különféle ellenfelektől kell megvédened a bázisod, tornyok és közelharci egységek segítségével. A telepítéshez egy játékban előre meghatározott valutát, azaz aranyat használsz, amit a megölt ellenséges egységek után kapsz, mivel minden ellenséges egység adott meghatározott mennyiségű aranyat ad a megsemmisítésük után. A játék győzelmének az a feltétele, hogy bizonyos számú hullámot kell túlélnie a játékos bázisának egy adott időkereten belül, anélkül, hogy az életeinek a száma nullára csökkenne.

2. A projekt célja

A fejlesztési folyamat elsődleges prioritása a csapattagok elméleti programozói, szoftverfejlesztői és tesztelői ismereteinek elmélyítése, valamint a csoportmunka során felmerülő projektmenedzsmenti feladatok és képességek éles helyzetben történő validálása volt. Fejlesztői szándékunk egy olyan magas élvezeti értékkel bíró, ugyanakkor szakmai szempontból is kihívást jelentő rendszer létrehozása, amely a Tower Defense műfaján keresztül vonja be a felhasználót egy olyan környezetbe, ahol a stratégiai befektetései és erőfeszítései hosszú távon is eredményesek maradnak. A játékunk megalkotásakor törekedtünk a mechanikai sokszínűségre és a különböző nehézségi szintek implementálására, hogy a szoftver minden játékos számára optimális és változatos élményt nyújtson. A projekt lehetőséget biztosított a modern fejlesztési munkafolyamatok és a hatékony együttműködés gyakorlati elsajátítására, miközben egy komplex, több platformot kiszolgáló játékrendszert hoztunk létre.

3. SWOT-elemzés

A projektünk analízisét SWOT-elemzéssel végeztük el. Erre szükségünk van, mert megmutatja nekünk azt, hogy milyen potenciális gyengeségek, fenyegetések és lehetőségek várják a munkánkat. Az üzletek világában is sokat használt SWOT-elemzés, azaz Strengths, Weaknesses, Opportunities and Threats-elemzés egy gyakran bevett stratégia marketingeseknek. A mi SWOT-unkat Excelben végeztük el. Az alábbi alfejezetekben láthatóak a projektünkről a saját és az AI által elvégzett SWOT-elemzés áttekintésének végeredményei.

3.1. Strengths (Erősségek)

- **Többplatformos jelenlét:** A `Unity` motor használata lehetővé teszi, hogy a játék asztali gépen és mobileszközökön is zökkenőmentesen fusson, szélesebb célközönséget elérve.
- **Integrált ökoszisztéma:** A `Vue 3` alapú weboldal és az `ASP.NET Core API` szoros egysége biztosítja a valós idejű adatszinkronizációt a játék és a felhasználói profil között.
- **Robusztus fejlődési rendszer:** A backend által vezérelt tapasztalati pont (XP) és szintlépési mechanika hosszú távú motivációt nyújt a játékosoknak.
- **Modern technológiai stekk: Az `Entity Framework Core`** és a `MySQL` használata biztonságos és skálázható adatkezelést tesz lehetővé a ranglisták és felhasználói adatok számára.
- **Egyedi vizuális világ:** A karakterek és tornyok kidolgozott "tulajdonság" rendszere mélységet ad a stratégiai tervezésnek.

3.2. Weaknesses (Gyengeségek)

- **Magas erőforrásigény:** A `Unity WebGL` buildjei jelentős memóriát igényelhetnek a böngészőben, ami régebbi hardvereken futtatási nehézségeket okozhat.
- **Kezdeti tartalomhiány:** Indie projektként a pályák és ellenségtípusok száma korlátozott lehet a nagy költségvetésű versenytársakhoz képest.

- **Hálózati függőség:** A fejlődési rendszer és a ranglista teljes mértékben az API elérhetőségétől függ, így offline módban a haladás nem menthető.

- **Komplex architektúra:** A három különböző technológia (`Unity, C# API, Vue 3`) összehangolása növeli a hibalehetőségek számát a fejlesztés során.

3.3. Opportunities (Lehetőségek)

- **Közösségi funkciók bővítése:** Lehetőség van klánrendszer, barátlisták vagy akár aszinkron multiplayer módok implementálására az adatbázis bővítésével.
- **Szezonális események:** Az API-n keresztül könnyen vezérelhetők időszakos kihívások vagy limitált ideig elérhető torony-kinézetek.
- **Oktatási potenciál:** A játékmechanika alkalmas lehet logikai vagy matematikai alapú kiegészítők befogadására, bővítve a felhasználói bázist.
- **Mobil applikációs áruházak:** A meglévő Unity alapok miatt a projekt viszonylag kis ráfordítással natív Android alkalmazássá alakítható.

3.4. Threats (Veszélyek)

- **Erős piaci verseny:** A Tower Defense műfaj telített, rengeteg ingyenes és magas minőségű alternatíva érhető el a játékpiacon.
- **Adatbiztonsági kockázatok:** Mint minden webes API-val rendelkező rendszer, a Fortivex is ki van téve az esetleges SQL injekciós vagy brute force támadásoknak.
- **Technológiai elavulás:** A használt keretrendszerek (pl. `Vue 3` verzióváltások) gyors frissülése inkompatibilitási problémákat szülhet a hosszú távú fenntartás során.
- **Felhasználói lemorzsolódás:** Ha a fejlődési görbe túl meredek vagy a szerverek lassúak, a játékosok gyorsan elhagyhatják a platformot.

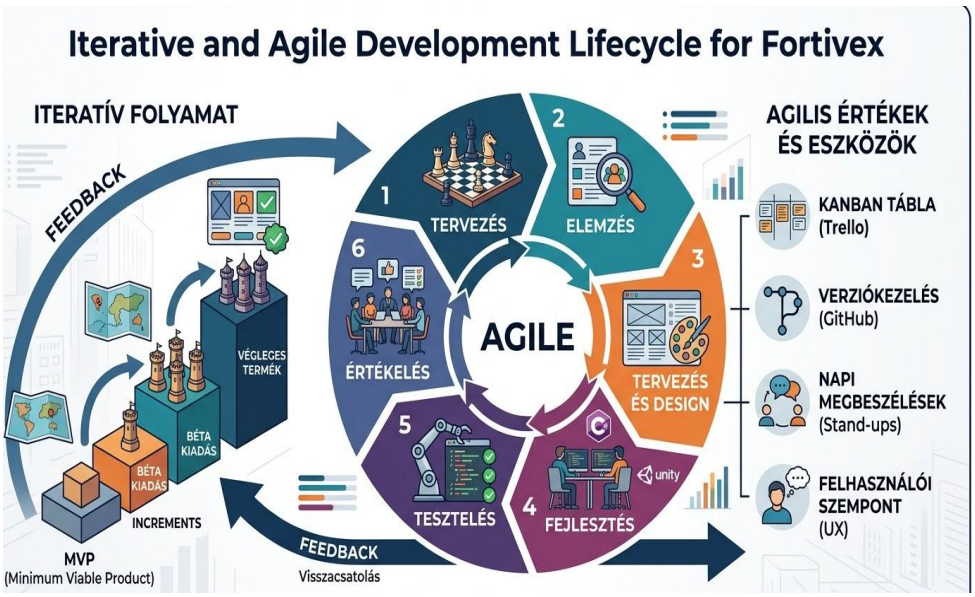
3.5. Fejlesztési modell

A Fortivex fejlesztése során egy hibrid megközelítést alkalmaztunk, amely ötvözi az **Agilis szoftverfejlesztés** rugalmasságát az **Iteratív modell** ciklikus fejlődésével. Választásunk azért esett ezekre a módszertanokra, mert egy ilyen komplex, több technológiai réteget

(`Unity,.NET, Vue 3 és Vite`) tömörítő projekt folyamatos visszacsatolást és gyors alkalmazkodóképességet igényel.

A fejlesztési folyamatot az alábbi elvek mentén strukturáltuk:

- **Agilis szemléletmód és Kanban:** A napi szintű munkaszervezésben az agilis alapelveket követtük, amelyeket a Trello felületén megvalósított Kanban-tábla segített vizualizálni. Ez lehetővé tette, hogy a csapat tagjai (asztali, Backend, Frontend szekciók) párhuzamosan, mégis összehangoltan haladjanak, elkerülve a fejlesztési szűk keresztmetszetek kialakulását.
- **Iteratív és inkrementális fejlesztés:** A projektet nem egyetlen, merev terv mentén valósítottuk meg, hanem több egymást követő ciklusban. Minden iteráció végén egy-egy működő részegységet (például egy alap pályát vagy egy működő bejelentkező felületet) hoztunk létre.
- **Prototípus-alapú tervezés:** Kiemelt figyelmet fordítottunk a korai prototípusok (`MVP – Minimum Viable Product`) létrehozására. Ez segített abban, hogy a `Unity`-ben megalkotott játékmechanikák és az API közötti kommunikációt már a fejlesztés korai szakaszában tesztelhessük, minimalizálva a későbbi integrációs kockázatokat.



1. ábra: Iteratív és Agilis fejlesztési modell

4. Játékmechanika és entitások

A játékunk mechanikája a klasszikus Tower Defense alapokból épül fel. Ezek általában ebből állnak össze:

1. A játékos kezdésnek egy teljes üres, lecsupaszított térképet kap, amin hamarosan egy hordányi ellenfél kezd el menetelni a bázisod felé.
2. A védelemhez pénz/arany kell és a felhasználó a játék elindításakor kezdeti alapnak kap valamennyi pénzt, amivel elkezdheti kiépíteni a védelmét. A továbbiakat viszont már önmagának kell elérnie. Aranyat úgy szerezhet, ha a védelme segítségével az ellenfelek életerejét nullára viszi és ezzel megöli őket. Különféle ellenfelek megölése különböző mennyiségű aranyat ad a játékosnak.
3. A játékos többféle toronyból választhat. Ez lehetőséget ad neki különféle stratégiákra, tervekre és arra, hogy a maximumot hozhassa ki a játékunk mechanikájából.
4. A játékos a játék menete során hozzáférést kap a tornyai fejlesztésére. Ezek felerősítik a tornyok alapstatisztikáit és új tulajdonságokkal.
5. A játék legtöbbször időre megy. Ez azért is fontos, mert a játékosnak nincs végtelen mennyiségű ideje eldönteni mit szeretne. Azt ott helyben kell eldönteni, szinte azonnal.

A következő alfejezetekben bemutatjuk azokat az entitásokat, amelyek a mi játékunk világának gerincét adják.

4.1.Ellenségtípusok és jellemzőik

A játékmenet során a felhasználó rengeteg különféle ellenséggel találja szembe magát, amiket a dokumentáció következő részében egyenként is górcső alá veszünk. Ezek az ellenfelek egy úgynevezett **„tulajdonság-rendszer”** alapján működnek. Ez a gyakorlatban azt jelenti, hogy minden egyes lény rendelkezik egyedi, vagy más egységekkel közös tulajdonságokkal, amik kifejezetten az adott csoportra jellemzőek. Egy-egy lénynek akár 3 vagy 4 ilyen különböző tulajdonsága is lehet egyszerre, ami meghatározza a viselkedését és a veszélyességét a pályán.

4.1.1.Goblinok

Elsőnek a fantasy alapú világok egyik legközismertebb, és talán legutálatosabb ellenfelét, a goblinokat mutatnánk be. Kicsik, zöldek, és erejük a létszámukban keresendő. A mi játékunkban is ezt vettük alapul. Egy hullám során 4-5 goblin érkezik a bázisod lerohanására, ún. “Csordák”-ban. Ezért a goblinok a “csordaszellem” tulajdonsággal vannak felruházva. Ez a tulajdonság a játék minden egyes térképén alkalmazva van. Az alábbi képek láthatóak a goblinokról a játékunkban. A két változat jelenléte nem csupán esztétikai választás, hanem taktikai mélységet is ad az első hullámoknak, mivel a dárdások képesek a védőegységeid hatótávolságán kívülről is sebet ejteni a bázis védőin. Éppen ezért a játékosnak már a korai szakaszban meg kell tanulnia a tornyok hatékony elosztását, hogy a nagy számban érkező, eltérő támadási stílusú csoportokat még azelőtt feltartóztassa, mielőtt a „csordaszellem” ereje áttörné a védvonalat.

2. ábra: Goblin dárdás és harcos játékbeli modellje

Mint ahogy látható, a játékunkban a goblinoknak kétféle variánsa is létezik. Ezekből az egyik a “Goblin Dárdás”, a másik pedig a “Goblin Harcos”, a tüskés buzogánnyal. Különbségekből egyetlen egy dolog van:a dárdás verziónak egy kicsivel magasabb a támadási hatótávja.

4.1.2. Ork

Az orkokat úgy illesztettük be a játékunkba, hogy az ilyen fantasy archetípusok közül egyet tökéletesen leképezzen. “A lassú, de vaskos” archetípust. Ez alapján az orkoknak alacsony mozgási sebességük van a játékban, viszont, ennek ellenében nagyon sok lövést kibír a tornyainkból. Az alábbi képen látható az ork. Tulajdonságok közül a “Szívósság”-ot kapta. Gyengeségének az “Óriásölő” fegyverek számítanak. Az óriásölő tornyokról a következő alfejezetben fogok írni. Az orkok „Szívósság” tulajdonsága komoly nyomás alá helyezi a védelmi vonalakat, mivel statisztikailag jóval több találatot képesek elviselni, mint a kisebb termetű ellenfelek. Lassú haladásuk ellenére veszélyességük abban rejlik, hogy képesek „élő pajzsként” felfogni a lövedékeket a mögöttük haladó gyorsabb egységek elől, így kényszerítve a játékost a tüzelési prioritások újragondolására. Megállításukhoz elengedhetetlen a nagy átütőerejű, kifejezetten páncélozott célpontok ellen tervezett tornyok korai telepítése, mielőtt a túlerő áttörné a kritikus pontokat.

3. ábra: Ork játékbeli modellje

4.1.3. Sámán

A következő egység egy újfajta mechanikát vezet be az ellenfelek részéről. Az ellenségnek innentől vannak gyógyító egységei a pályán. Ezek az ellenségek a sámánok. Lassan mozognak, rendkívül magas életerővel rendelkeznek, és amíg életben vannak, időszakosan gyógyítják az ellenségeket. Emiatt, hogy ne legyenek az ellenfeleid megölhetetlenek, a legelsők között kell a sámánokat kiiktatnia a játékosnak. A sámánnak néha lespawnolhat egy ritka, de még erősebb verziója, a Voodoo, ami nem csak egy célpontot, hanem egy adott célterületen belül az összes ellenséget képes gyógyítani.

4. ábra: Sámán és a Voodoo játékbeli modelljei

4.1.4. Farkasok

A játékmenet során, viszonylag korán találkozik a játékos farkasokkal. A farkasok a mi világunkban két dologról ismertek. Sokan vannak és nagyon gyorsak. Két tulajdonsággal illettük meg őket. Az egyik a goblinoktól már korábban ismert *"csordaszellem"* a másik egy új egyedi tulajdonság, az *"eluzivitás"*. Az *eluzivitás*-t az angol "*elusive*" szóból hoztuk át és nehezen hozzáférhetőt jelent. Ez a játékunkban azt jelenti, hogy a gyorsasága miatt nagyon könnyedén tud a tornyok támadási hatótávjából kikerülni. A játékos számára a farkasokra külön figyelni kell, mivel ők okozhatják az egyik legnagyobb fenyegetést a játék győzelmi feltételeire nézve. A farkasok elleni védekezés kulcsa a folyamatos lassítás és a területi sebzés kombinálása, mivel egyetlen torony ritkán képes végezni a teljes falkával azok extrém mozgási sebessége miatt. Ha a játékos nem telepít stratégiailag fontos pontokra lassító mechanizmusokat, a farkasok „Eluzivitása" révén pillanatok alatt átszakíthatják a védvonalat, közvetlen veszélybe sodorva a bázis életerejét.

5. ábra: A farkas játékbeli modellje

4.1.5. Denevérek

A játékunk közepétől megjelenik egy új egyedi képességgel rendelkező egység. Ezek a denevérek. Ha a farkasok fenyegetést jelentettek, akkor a denevérek ezt megsokszorozzák. A denevérek gyorsak, emellett *"repülő"* egységként érzékeli a játékunk őket, ezért a közelharci tornyok egyszerűen nem tudják megtámadni. A gyors mozgásuk miatt szintúgy ki tudnak kerülni a hatótávból. Érdemes ellenük több gyorsan tüzelő tornyot felszerelni a bázisodra.

6. ábra: Denevér játékbeli modellje

4.1.6. Monstrum tank

A játékunk legutolsó ellenfele. A Monstrum tank egy goblinok és orkok által összetákolt harci gépezet, ami lassú menetével egy folyamatos fenyegetést jelent a játékos számára, amíg ez az ellenfél a képernyőn van. Végső ellenséghez méltóan különleges képességekkel is felruháztuk. A Tank az ellenség szemszögéből nézve egy védelmező, támogató egység. Képes egy aktív képességre, ami a "Füstalak" nevet viseli. A füstalak egy füstbombát dob le a földre, amiben a tank garantáltan és a körülötte levő ellenfelek is megcélozhatatlanok lesznek összesen két másodpercre. Ez azt jelenti, hogy két másodpercen keresztül teljesen haszontalanná teszi a tornyaidat és szabadon mehet előre. Másik képessége még a rendkívül magas életereje. Ez az ellenség kizárólag az utolsó pályán és annak is a legutolsó hullámában bukkan fel. Készülj fel a védelemre ellene.

7. ábra: A Monstrum tank játékbeli modellje

4.2. Toronyrendszer és stratégiai lehetőségek

A szörnyek ellenkezőjéül most tekintsük át, mit tud a játékos tenni ennyi veszélyes ellenfél ellen.
A játékos a pálya elindításakor aranyat kap. Egy tornyhelyre kattintva egy 5 lyukas “tárcsa” jelenik meg és ezekből tud a játékos választani, melyiket építse fel a védelemre. Körülbelül 2-3 kezdeti toronyra futja a legelején. Ahogy jönnek az ellenfelek, a játékos ölésenként 1 aranyat kap. Bőven van annyi ellenfél, amivel egy teljes pályát ki lehet építeni tornyokkal. A második pályától kezdve, a kezdeti ár duplájáért a játékos képes 2 fejlesztést is rakni mindegyik toronyra, amivel megváltozik a megjelenésük, erősebb statokkal és új képességgel bővülnek.
Akkor tartsunk egy szemlét abból, amit játék kínálni tud nekünk védelemből.

Persze az arany nem csak úgy az ölünkbe hullik, okosan kell vele gazdálkodni, mert a keret véges. Az elején még mindenki kapkod, próbálja kitalálni, hova is tegye azt a pár tornyot, amire futja, de ahogy dőlnek a goblinok, úgy kezd el hízni az erszényünk is. Tulajdonképpen a játék rákényszerít arra, hogy folyamatosan figyeljük a pályát: ha nem építünk, áttörnek, ha rossz helyre építünk, elszórjuk a pénzt.

Nagy könnyebbséget jelent, hogy a második pályától már nem csak az alap tornyokkal kell beérnünk. Igaz, hogy mélyebben a zsebünkbe kell nyúlni értük, de a fejlesztések után a tornyaink nemcsak komolyabban néznek ki, hanem tényleg odacsapnak. Ilyenkor jön meg az ember önbizalma, amikor látja, hogy a korábban problémát okozó farkasok vagy orkok már nem jutnak el a bázisig. Ez a fejlődési lehetőség adja meg a játék igazi ízét, hiszen mindenki szereti látni, ahogy az üres tisztásból egy komoly védelmi rendszer válik.

Most pedig nézzük meg egyesével ezeket a védelmi vonalakat, mert mindegyiknek megvan a maga kis trükkje és haszna, amivel kisegíthet minket a bajban.

4.2.1.Közelharci tornyok

Az első torony, amivel a játékos találkozhat, az a közelharci torony. Ez lesz a legmeggyőzőbb a korai szakasz során, mivel a legolcsóbb toronyként csak 4 aranyába kerül a felhasználónak. A torony, amint megvette a játékos, közelharcos katonákat küld ki maga köré, hogy azok harcoljanak a szörnyek ellen. Egy toronyból maximum két katona harcolhat kint egyszerre. Nincs számkorlát, hogy hány ilyen tornyot rakhatsz le, így megnyílik a kapu a stratégiákra.

Egyik példa, amit én találtam, az egy "közrefogás" stratégia, amivel a katonákat a bázisod közepébe telepíted, köré pedig a távolsági tornyok vannak. A harcosaid jó pár ütést kibírnak. A fegyverforgatóid csak a torony hatótávján belül tudnak harcolni, de egyéni hatótávjuk is van. Fejlesztéskor a katonák lándzsás harcosokká válnak, amivel több védelmük és nagyobb támadási hatótávjuk lesz, a második fejlesztés során pedig ágyús katonákká válnak, növelve a sebzésüket és még tovább a hatótávjukat. Az alábbi képeken látható, hogy változik a fejlődés során a katonák külseje.

8. ábra: A gyalogos játékbeli modelljei és fejlődései

4.2.2. Íjásztorony

A következő toronytól kezdődően minden tornyuk távolsági lesz. Az íjásztorony egy 5 aranyba kerül egység, amit lerakva egy íjász kezdi el a közelébe jövő egységeket támadni. A támadási sebessége átlagos, sebzése átlagos. Fejlesztések során növekszik a hatótávja, harmadik során pedig a sebzése.

9. ábra: Az íjásztorony játékbeli modelljei és fejlődései

4.2.3. Mágustorony

A következő torony a mágusok tornya. Ez a torony, magas hatótávval rendelkezik és magas sebzéssel, de a tüzelési gyorsasága viszonylag lassú. Mágikusan sebez. Fejlesztés során nő a támadási sebessége, harmadik szinten pedig egy aktív varázslatot is képes elsütni. Minden varázslótorony képes elsütni ugyanazt a varázslatot, ha több van fejlesztve.

10. ábra: A mágustorony játékbeli modelljei és fejlődései

4.2.4. Bombázó Torony

A negyedik torony az ágyús torony, amelynek több egyedi funkciója is van. Ez az egyedüli torony, amely képes egynél több célpontot is egyidőben sebezni, emellett nagyon magas hatótávja, sebzése és nagyon lassú támadási sebessége van. Ez a torony nem képes célbavenni a repülő ellenfeleket. A torony egy ún. “Óriásölő” effekttel van felruházva, ami azt csinálja, hogy a magas életerejű, az az több, mint 10 pontnyi életerejű szörnyekbe az alapsebzésén túl extra 5%-ot hozzáad a sebzéséhez. Az óriásölőre az orkok, sámánok, voodook és a tank érzékeny. Fejlesztések során a torony sebzése, hatótávja és kissé a támadási sebessége nő. A Bombázó torony igazi ereje a stratégiai pontokon mutatkozik meg, ahol a szűk keresztmetszeteknél feltorlódó ellenséges csoportokat egyetlen lövéssel képes megtizedelni. Bár a repülő egységek elleni védtelensége komoly kockázatot jelent, az „Óriásölő” effektus nélkülözhetetlenné teszi a kései hullámok során, amikor a hatalmas életerővel rendelkező szörnyek már csak ilyen koncentrált tűzerővel állíthatók meg. Jól kiegészíti a gyorsabb tornyokat, hiszen amíg azok a kisebb egységeket takarítják be, az ágyú zavartalanul koncentrálhat a legveszélyesebb, páncélozott célpontok lassú, de biztos felőrlésére.

11. ábra: A bombázó torony játékbeli modelljei és fejlődései

4.2.5.Dárdavető torony

A dárdavető torony a játékos egyik szuperfegyvere a nehezen megölhető ellenfelek ellen. Ez hasonlóan az íjászokhoz, egy egycélpontos torony, de annál effektívebb a jó ellenféllel szemben. Magas támadási sebessége, nagyon magas sebzése, de aránylag kis hatótávja van. Óriásölő effekttel rendelkezik, emiatt a nagy életerejű ellenfeleket könnyedén meg tudja törni. Fejlesztések során növekszik kicsit a hatótávja és a sebzése. A stratégiai elhelyezése kulcsfontosságú, hiszen viszonylag rövid hatótávolsága miatt leginkább a kanyarokba vagy a gyalogos egységek által feltartóztatott ellenséges gócpontok mellé érdemes telepíteni. Mivel a lövedékei páncéltörő jellegűek, ez a torony válik a védelem tartóoszlopává a későbbi hullámok során érkező főnök-típusú szörnyekkel szemben. Megfelelő fejlesztéssel és pozicionálással képes a legszívósabb tankokat is meggyengíteni, mielőtt azok elérnék a bázisunk kapuját. Ez a specializált tűzerő teszi lehetővé, hogy a játékos ne csak a mennyiségi, hanem a minőségi fenyegetésekre is hatékony választ adhasson a harctéren.

12. ábra: A dárdavető torony játékbeli modelljei és fejlődései

4.2.6.Varázslatok

A varázslatok bár nem sorolhatók a hagyományos értelemben vett toronyrendszerbe, mégis elengedhetetlen stratégiai eszköztárat biztosítanak a játékos számára a második pályától kezdődően. Ezek a különleges képességek olyan kritikus pillanatokban vethetők be, amikor a tornyok alapvető tűzereje már nem elegendő a horda megfékezésére, így egyfajta „vészhelyzeti” beavatkozásként szolgálnak. Az alábbiakban a játék során tapasztalt megjelenési sorrendjükben mutatjuk be ezt a két meghatározó mágikus erőt.

Csillaghullás:

A játékos ezen a szinten kapja meg az első közvetlen taktikai képességét, a Csillaghullást. Ez a varázslat lehetővé teszi a közvetlen interakciót a csatatérrel: a játékos kijelölésére egy pusztító asztrális energia csapódik le az égből, csapást mérve, azonnali és kritikus sebzést okozva a fenyegetést jelentő ellenfélre.

13. ábra: A Csillaghullás játékbeli ikonja

Meteorraj:

Az utolsó pályán használható kizárólag. Gombnyomásra a megadott területre 3 másodpercen keresztül meteorraj csapódik be, folyamatosan magas sebzést okozva a benne lévő ellenfeleknek. Pusztító ereje miatt ez a végső fegyver a hatalmas életerővel rendelkező főellenségek és a sűrű ellenséges oszlopok ellen, ahol a területi sebzés maximálisan érvényesülni tud. Mivel a hatása több másodpercig tart, stratégiai fontosságú a lassító tornyok közelében elsütni, hogy az áldozatok a lehető legtovább a becsapódási zónában maradjanak.

14. ábra: A Meteorraj játékbeli ikonja

4.3. Játékmenet a gyakorlatban

Az alkalmazás megnyitása után a felhasználónak először **be kell jelentkeznie** a fiókjába. Sikeres bejelentkezést követően a főképernyőn megjelenő **Start gombra** kattintva a játékos automatikusan a pályaválasztó felületre kerül.

15. ábra: Belépés/Regisztráció

16. ábra: Start menü

Új felhasználóként kezdetben csak az **első pálya** érhető el, a további pályák fokozatosan oldhatók fel a játék előrehaladtával.

17. ábra: Pályakiválasztási menü

A nyilakra kattintva tudsz navigálni a pályák között. Három szintből tudsz választani:

- Első szint: **Elvarázsolt erdő**
- Második szint: **Sötét kanyon**
- Harmadik szint: **Csendes part**.

A pipa gombra kattintva lehet a játékot elindítani.

Első pálya: Elvarázsolt erdő

Az első pályán csak venni tudsz tornyokat, további fejlesztések korlátozottak, a következő pályától kezdődően tudod fejleszteni a tornyaidat.

18. ábra: Az első pálya kinézete

A játék menete rendkívül egyszerű és intuitív. A pályán elhelyezett **kis táblákra kattintva** a játékos tornyokat vásárolhat, amelyek segítségével védekezhet az érkező ellenségek ellen. Minden egyes torony megvásárlásához bizonyos mennyiségű **aranyra** van szükség, ezért érdemes átgondoltan gazdálkodni az erőforrásokkal.

19. ábra: Toronyvásárlás

Minden sikeresen legyőzött ellenség után a játékos **aranyat** kap jutalmul, amelyet a játék során újabb tornyok vásárlására fordíthat. Minél több ellenséget győz le, annál több tornyot építhet ki, ezzel erősítve védelmi vonalát a következő hullámok ellen.

A játék során a képernyőn folyamatosan látható egy **HUD**, amely minden fontos információt megjelenít a játékos számára, hogy mindig naprakész legyen a játék állásáról.

A képernyőn **arany színnel** jelenik meg a játékos aktuális **pénzegyenlege**, így mindig könnyen nyomon követhető, hogy mennyi arany áll rendelkezésre a következő torony megvásárlásához.

A képernyő közepén található a **körök számlálója**, amely megmutatja, hogy még hány kör van hátra a pálya teljesítéséhez. Ez segít a játékosnak megtervezni a védekezési stratégiáját a hátralévő hullámokra.

A játékos **életerejét** egy piros színű jelző mutatja, amely folyamatosan frissül attól függően, hogy mennyi ellenségnek sikerült átjutnia a védelmi vonalon. Ha az életerő nullára csökken, a játék véget ér.

Végül a **homokóra ikon** jelzi, hogy mennyi idő van még hátra a következő ellenséges hullám érkezéséig. Ezt figyelembe véve a játékos felkészülhet, és még időben elhelyezheti a szükséges tornyokat a pályán.

Második pálya: A Sötét Kanyon

A Fortivex második pályája, a **Sötét Kanyon**, jelentős mérföldkő a játékos előrehaladásában, hiszen itt találkozik először a komplexebb stratégiai mélységekkel és az erőforrás-menedzsment valódi súlyával. A vizuális kontraszt azonnal érezhető: amint a játékos jóváhagyja a pálya indítását, a korábbi sivárabb tónusok után egy élénkzöld, látszólag tiszta és érintetlen erdőben találja magát. Ez a békés környezet viszont meglehetősen csalóka, hiszen a kanyon mélyéről érkező fenyegetés minden eddiginél intenzívebb és szervezettebb. A játékmenet dinamikája ezen a térképen alapjaiban változik meg.

- **Sokszínű ellenséges hullámok:** Az első hullám kezdetétől fogva a rendszer nem hagy időt a pihenésre. A gyengébb goblinok mellett már párosával érkeznek a robusztus orkok, és a gyors mozgású farkasok. A kör előrehaladtával a taktikai paletta tovább bővül: a légtérben megjelenő denevérek a távolsági védelmet tesztelik, míg a hátul megbújó sámánok misztikus erejükkel támogatják a lerohanó hordát, kényszerítve a játékost a célpontok priorizálására.
- **Fejlesztési rendszer:** A Sötét Kanyon legfontosabb újítása a tornyok továbbfejlesztésének lehetősége. A védelem kiépítése közben feltűnő új opciók lehetővé teszik, hogy a meglévő bázisokat ne csak darabszámban, hanem hatékonyságban is növeljük. Ez kulcsfontosságú, hiszen az időzítő kíméletlenül, percenként zúdítja az újabb és újabb hordákat a bázisra, ahol a nyers erő már nem elegendő a győzelemhez.
- **Magas fokozatú arzenál:** A pálya nehézsége megköveteli a drágább, speciális egységek korai bevetését. A dárdavetők és a területre ható (AoE- Area of Effect) sebzést biztosító bombázó tornyok stratégiai elhelyezése jelenti a különbséget a bázis eleste és a sikeres védekezés között.
- **Csillaghullás varázslat:** A játékos ezen a szinten kapja meg az első közvetlen taktikai képességét, a Csillaghullást. Ez a varázslat lehetővé teszi a közvetlen interakciót a csatatérrel: a játékos kijelölésére egy pusztító asztrális energia csapódik le az égből, csapást mérve a legkritikusabb azonnali és kritikus sebzést okozva a fenyegetést jelentő ellenfélre. Ez az elem beemeli a játékmenetbe a gyors reakcióidő fontosságát, megtörve a statikus védekezés rutinját. A Sötét Kanyon így nem csupán egy újabb helyszín, hanem egy taktikai próbatétel, ahol a játékosnak össze kell hangolnia a fejlesztéseket, a speciális tornyok erejét és a mágikus támogatást a győzelem érdekében.

Harmadik pálya: A csendes part

A harmadik fejezet, a **Csendes part**, a Fortivex kampányának egyik leglátványosabb és stratégiailag legösszetettebb helyszíne. A vizuális környezet itt egy békés, melankolikus őszi tájat idéz: a vöröses-narancssárga falevelek, a tükörsima kis tavak és a kerítéssel elkerített, hánykolódó csónakú tengerparti szakasz egyfajta hamis biztonságérzetet sugall. Ez a nyugalom azonban gyorsan szertefoszlik, amint a játékos szembesül a térkép egyedi kihívásaival.A harmadik szint mechanikai és taktikai újításai a következők:

Ez az első olyan helyszín, ahol a játékosnak **két fronton** kell helytállnia. Az ellenséges hordák már nemcsak a megszokott nyugati (bal) oldalról érkeznek, hanem a keleti (jobb) irányból is egyszerre indítanak támadást. Ez a változtatás alapjaiban írja felül az addigi stratégiákat, hiszen a védelmi erőforrásokat (pénz és tornyok) okosan kell megosztani a két bejárat között, elkerülve, hogy bármelyik útvonalon áttörjenek az egységek.

A gyengébb goblinok száma ezen a szinten drasztikusan lecsökken, helyüket a szívósabb orkok, gyors farkasfalkák és támogató sámánok veszik át. A legnagyobb veszélyt azonban a Voodoo megjelenése jelenti, amely a repülő denevérekkel karöltve a pálya korai és középső szakaszának legkritikusabb eleme. A két irányból egyszerre érkező, magas életerejű és mágikus képességekkel rendelkező egységek folyamatos nyomás alatt tartják a védvonalat.

A növekvő nyomás ellensúlyozására a játékos egy újabb pusztító képességet kap: a **Meteorrajt**. Ellentétben a Csillaghullással, ez a varázslat egy meghatározott területet bombáz folyamatosan 3 másodpercen keresztül. Ez a képesség kulcsfontosságú a két irányból érkező ellenséges csoportosulások megtisztítására, ahol a folyamatos területre ható sebzés képes megállítani a túlerőt.

A Csendes part teszi elérhetővé a tornyok **harmadik, legmagasabb fejlesztési szintjét**. Ez nem csupán statisztikai növekedést (sebzés, hatótáv) jelent, hanem minden toronytípus egyedi, passzív vagy aktív képességgel bővül:

- **Közelharci torony:** A kiképzett egységek öngyógyító képességet kapnak, így sokkal tovább képesek feltartóztatni az ellenséget a frontvonalon.

20. ábra: Közelharci torony képessége

- **Íjásztorony:** Megnyílik a „**Nyílzápor**” lehetőség, amely sűrű sorozatokkal árasztja el a kijelölt területet.
- **Dárdavető:** Egy speciális hálóvető mechanizmussal egészül ki, amely képes rövid időre teljesen lelassítani vagy helyben tartani a legveszélyesebb célpontokat, időt nyerve a többi toronynak a sebzéskifejtésre.
- **Mágustorony:** A legmisztikusabb fejlesztés, amely lehetővé teszi a **Kényszerteleport** használatát. A mágus képes egyetlen kiválasztott ellenséget a tér tetszőleges pontjára teleportálni, ezzel értékes időt nyerve a védelemnek, vagy visszaküldve a veszélyes egységet a tűzerő középpontjába.
- **Bombázótorony:** A pusztító erejű ágyú egy speciális **Kábítólövedéket** kap az arzenáljába. Bár a lövedék egyszerre csak egyetlen célpontot érint, a becsapódás ereje

képes rövid időre teljesen megbénítani a legerősebb ellenfeleket is, megszakítva azok támadását vagy képességeit.

A **Csendes part** küzdelmeinek abszolút csúcspontja és a Fortivex kampányának lezárása a **Monstrum Tank** megjelenése. Ez a gigantikus, vasszerkezetű monstrum a 9. hullám során gördül be a csatatérre, jelezve a játék végső összecsapásának kezdetét. Mint a játék „Final Boss”-a, a harckocsi minden eddigi ellenségnél nagyobb kihívás elé állítja a játékost: mozgása ugyan végletesen lassú, de szinte elképesztő szívóssággal és életerővel rendelkezik, amely ellen a legmagasabb szintű toronyfejlesztések összehangolt tüze is csak fokozatosan tud fellépni.

A harckocsi taktikai veszélyességét tovább növeli, hogy nem egyedül érkezik; a pályát elárasztó egységek sűrű gyűrűje veszi körül, amelyek élő pajzsként védik a monstrumot a közvetlen találatoktól. A csata legkritikusabb eleme azonban a tank speciális védelmi mechanizmusa, a **„Füstalak”** képesség. Nagyjából 30 másodpercenként a harckocsi egy sűrű ködöt árasztó gránátot dob maga alá, amely rövid időre teljesen elrejti őt és a közvetlen közelében tartózkodó kíséretét.

Ebben a füstfelhőben a Monstrum Tank és csapatai 2 másodpercre megcélozhatatlanná válnak a tornyok számára, megszakítva minden folyamatos sebzést és tüzelési sorozatot. Ez a rövid időablak stratégiai szempontból döntő: a játékosnak pontosan kell időzítenie a **Csillaghullás** vagy a **Meteorraj** varázslatokat, hogy ne pazarolja el azokat a láthatatlanság ideje alatt. A győzelemhez elengedhetetlen a harmadik szintű toronyképességek – különösen a mágus teleportációjának és a bombázó kábítólövedékének – mesteri használata. Amint a Monstrum Tank páncélzata végleg megadja magát és a monstrum elpusztul, a térkép és vele együtt a hadjárat is véget ér. A győzelem pillanata nemcsak a vizuális effektusokban merül ki, hanem a játékos számára egyfajta megkönnyebbülést is jelent a hosszú percek óta tartó feszített koncentráció után. A tank pusztulását követően a maradék ellenséges egységek fejvesztve menekülni kezdenek, végleg szabaddá téve az utat a bázisunk előtt. Ez a momentum szolgál a kampány érzelmi csúcspontjaként, ahol a korábban befektetett arany és a gondosan megtervezett stratégia végül kifizetődik. A képernyőn megjelenő győzelmi felirat nemcsak egy pálya végét jelzi, hanem igazolja a játékos taktikai rátermettségét is a Fortivex legnehezebb kihívásával szemben. Ezzel a végső csatával zárul le a történet íve, lehetőséget adva a felhasználónak, hogy büszkén tekintsen végig az általa felépített, áttörhetetlen védelmi rendszeren.

21. ábra: A 9.ik kör végén a tank megjelenik

5. Backend és fejlődési rendszer

A Fortivex ökoszisztéma motorja egy modern, `ASP.NET Core` alapokon nyugvó `RESTful API`, amelyet kifejezetten a nagy teherbírás és a biztonságos adatkezelés szempontjai szerint terveztünk. Az architektúra technikai magvát a `FortivexContext` alkotja; ez a komponens az **`Entity Framework Core`** technológiát használva hidalja át a szakadékot a programkód és az adattároló réteg között.

A rendszer logikai felépítése során olyan dedikált adatmodelleket alakítottunk ki, mint az `Accounts`, a `PlayerStats` és a `PlayerProgress`. Ezek az entitások szigorú sémák mentén illeszkednek az adatbázis-struktúrába, ami garanciát jelent az adatok állandóságára és a profilok közötti integritás megőrzésére. Ez a rétegelt megközelítés lehetővé teszi, hogy a játékosok előmenetele és statisztikái minden körülmények között pontosan és gyorsan elérhetőek legyenek a kliensoldal számára.

5.1. Szintezési rendszer: Tapasztalati pontok és szintek kezelése

A fejlődési rendszer technikai alapját a `PlayerStatsController` és a `PlayerProgressController` finomhangolt végpontjai alkotják, amelyek lehetővé teszik a játékmenet során keletkező adatok, például a tapasztalati pontok valós idejű szinkronizációját.

Az adatok feldolgozása során a `UserService` réteg különíti el az üzleti logikát a hálózati rétegtől, ami garantálja a szintlépési kalkulációk pontosságát és megkönnyíti az egységtesztelési folyamatokat. Minden adatátvitel `DTO` (Data Transfer Object) minták segítségével valósul meg, ami szétválasztja a belső adatmodelleket a publikus interfésztől, minimalizálva az adatszivárgás kockázatát és optimalizálva a hálózati forgalmat.

5.2. Teljesítmény rendszer: Teljesítmények és mérföldkövek

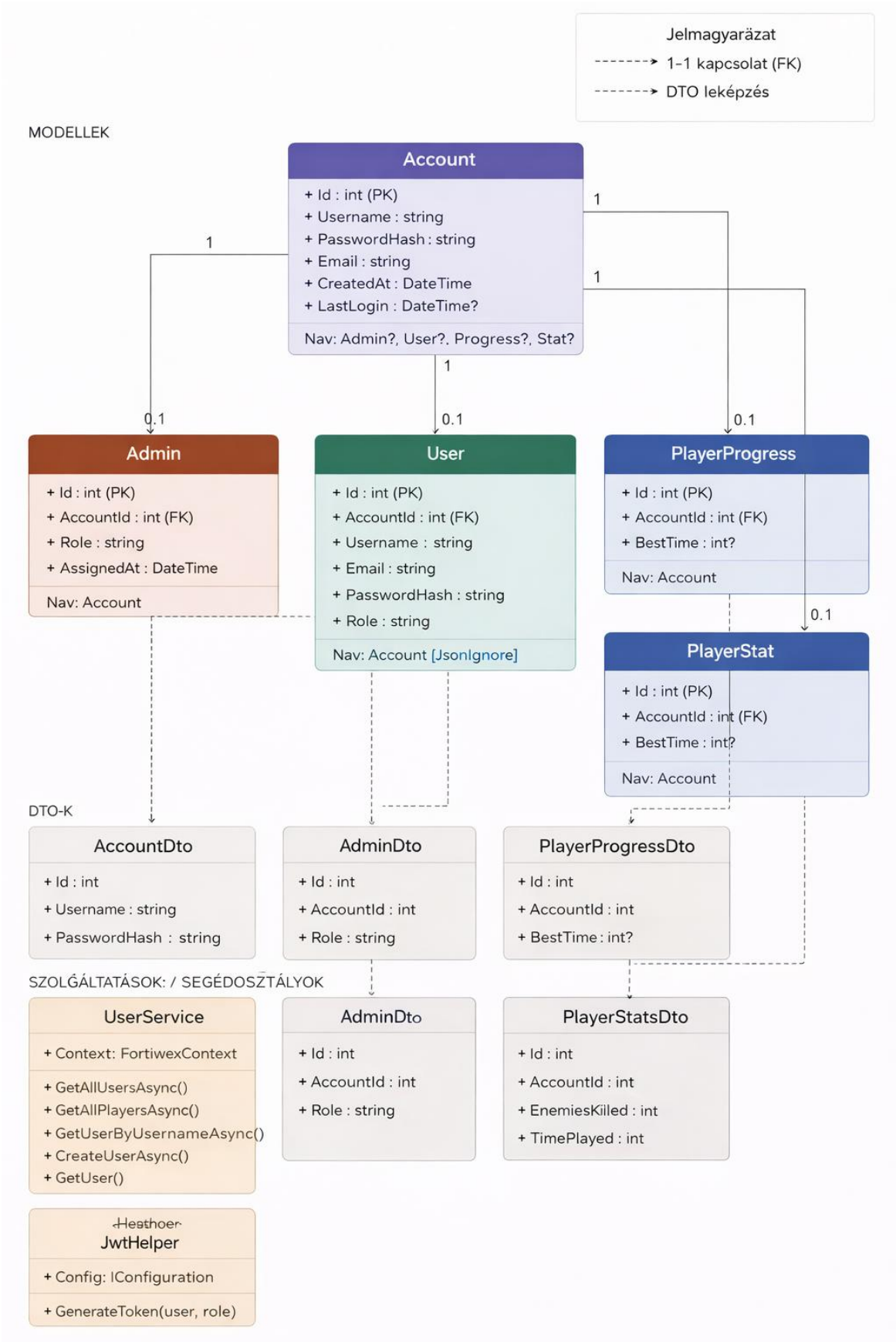
A mérföldkövek és eredmények biztonságos rögzítéséért a backend biztonsági rétege felel, amelynek gerincét a `JwtHelper` osztály adja. Ez az iparági sztenderdnek számító `JSON Web Token` technológiát alkalmazza az állapotmentes hitelesítéshez és autorizációhoz, megakadályozva az eredmények illetéktelen módosítását.

Ez a struktúra nem csupán a jelenlegi funkciók kiszolgálására alkalmas, hanem rugalmas alapot biztosít a jövőbeli bővítésekhez, legyen szó bonyolultabb teljesítmény-logikákról vagy analitikai lekérdezésekről.

6. Rendszerarchitektúra és alkalmazott technológiák

A Fortivex fejlesztésekor egy olyan többszintű rendszert hoztunk létre, amely maradéktalanul megfelel a mai elvárásoknak: bárhonnan elérhető, biztonságosan kezeli az adatokat és gyors válaszidőt biztosít. A tervezés során a modularitásra, azaz az építőkockaszerű felépítésre törekedtünk. Ennek köszönhetően élesen elválasztottuk egymástól a látványvilágért felelős megjelenítő réteget a háttérben futó logikai folyamatoktól és az adattárolástól. Ez a megközelítés nemcsak stabilabbá teszi a szoftvert, hanem lehetővé teszi, hogy az egyes részeket egymástól függetlenül is fejleszthessük vagy javíthassuk.

Az architektúra három fő pillérre támaszkodik: `Unity` motorra épülő interaktív játékra, az `ASP.NET Core` alapú `REST API`-ra, valamint a `Vue 3 + Vite` keretrendszerrel megvalósított webes portálra. Ebben a fejezetben részletesen bemutatjuk ezeket a technológiai választásokat, kitérve az adatbázis-struktúra mélységeire és azokra a szoftveres megoldásokra, amelyek lehetővé teszik a zökkenőmentes kommunikációt a különböző platformok között. A választott technológiai stukk az egyes komponensek közötti szoros, mégis rugalmas integrációt szolgálja, ahol minden réteg a saját szakterületén nyújt maximális teljesítményt. Míg a `Unity` motor a vizuális élményért és a valós idejű interakciókért felel, az `ASP.NET Core` háttérrendszer biztosítja a skálázható és biztonságos adatkezelést a kérések kiszolgálása során. A `Vue 3` és `Vite` párosa egy rendkívül gyors és modern adminisztrációs felületet kínál, amely közvetlen rálátást biztosít a rendszer belső folyamataira és a felhasználói statisztikákra. Ez a rétegelt felépítés nem csupán a fejlesztési fázisban könnyítette meg a hibakeresést és a moduláris tesztelést, hanem hosszú távon is fenntartható és továbbfejleszthető alapot teremtett a projekt számára. A végeredmény egy olyan egységes ökoszisztéma, amelyben az adatok áramlása és a szolgáltatások elérhetősége transzparens módon valósul meg a végfelhasználók és a fejlesztők számára egyaránt.



22. ábra: Component diagram

6.1. Multiplatform megvalósítás: PC (Desktop) és Mobil (Android)

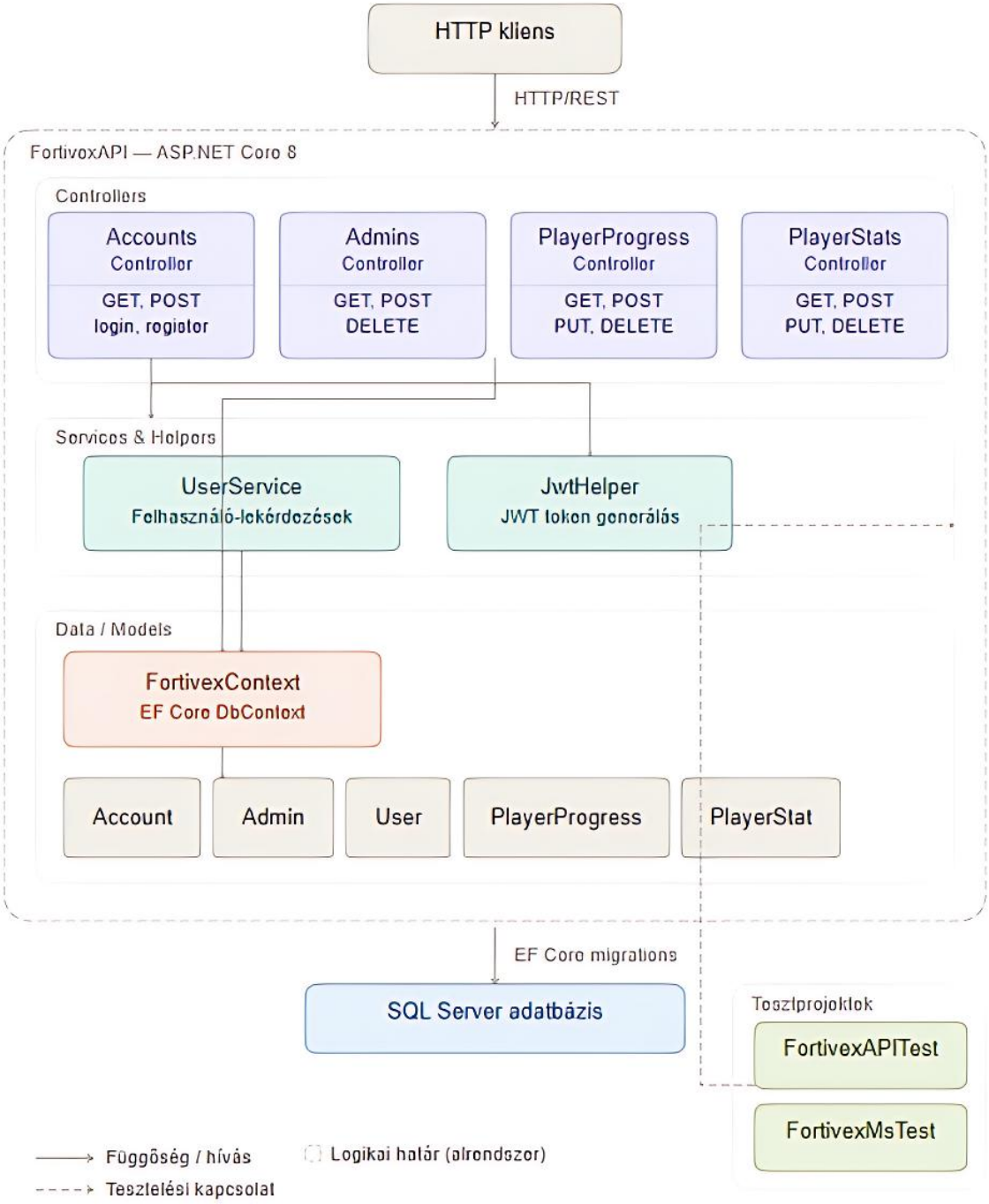
A Fortivex fejlesztése során kiemelt szempont volt a rugalmasság, ezért a `Unity 6.2` játékmotort választottuk alapul. Ez a technológia lehetővé teszi, hogy ugyanazt a kódbázist használva több különböző platformra is optimalizáljuk a játékot. A PC-s verzió esetében a hangsúly a stabil teljesítményen és a precíz, egérrel történő irányításon van, míg az Android platformra szánt kiadásnál a reszponzív UI és az érintőképernyős vezérlés kapott főszerepet. A keresztplatformos működés érdekében a `Unity` beépített fordítóit és optimalizációs eszközeit alkalmaztuk. A grafikai erőforrásokat és az irányítási sémát úgy alakítottuk ki, hogy a játékélmény konzisztens maradjon, függetlenül attól, hogy a felhasználó asztali gépen vagy egy hordozható okoseszközön játszik. Ez a megközelítés nemcsak a fejlesztési időt rövidítette le, hanem a későbbi frissítések és hibajavítások kezelését is jelentősen leegyszerűsíti.

6.2.Adatbázis-felépítés és adatok perzisztenciája

A játék adatainak biztonságos és tartós tárolásáért egy `MySQL` alapú relációs adatbázis felel. Az adatok perzisztenciáját, vagyis az állapotok megőrzését egy `ASP.NET` Core keretrendszerben megírt `REST API` biztosítja. Ez a réteg szolgál hídként a játék és az adatbázis között: a `Unity` kliens HTTP kéréseken keresztül kommunikál a szerverrel, így mentve el a játékosok előrehaladását, az elért pontszámokat és a megszerzett fejlesztéseket.

Az adatbázis-architektúra tervezésekor a normalizáltságra és a gyors lekérdezhetőségre törekedtünk. A legfontosabb entitások közé tartoznak:

- **Felhasználók (Users):** A regisztrációhoz és belépéshez szükséges hitelesítési adatok.
- **Statisztikák (Statistics):** A lejátszott meccsek száma, a Kill számláló állása és a legjobb túlélési idők.
- **Eredménytábla (Leaderboard):** A globális rangsor felállításához szükséges adatok összesítése.



23. ábra: UML Diagram

6.3. Frontend: Webes adminisztrációs felület és főoldal

A projekt kiegészítő részét képezi egy modern, `Vue 3` és `Vite` keretrendszerrel készült weboldal. Ez a felület kettős célt szolgál: egyrészt egy vonzó kezdőlapként (homepage) funkcionál az új érdeklődők számára, másrészt egy zárt, bejelentkezéshez kötött adminisztrációs és profilkezelő felületet biztosít a már regisztrált játékosoknak. A `Vue 3` reaktív természetének köszönhetően az oldalak villámgyorsan betöltődnek, a felhasználói élmény pedig gördülékeny marad.

A webes frontend közvetlenül ugyanahhoz az `API`-hoz kapcsolódik, mint a játék, így a játékosok a böngészőből is valós időben követhetik nyomon a saját vagy barátaik statisztikáit.

24. ábra: A játék főoldala

Az adminisztrációs felületen a felhasználók módosíthatják profiladataikat, megtekinthetik korábbi teljesítményeiket és hozzáférhetnek a játékkal kapcsolatos legfrissebb információkhoz. Ez az integrált webes jelenlét teszi teljessé a Fortivex ökoszisztémáját, hidat képezve a játékmenet és a közösségi interakciók között.

Admin Panel – Játékosok kezelése

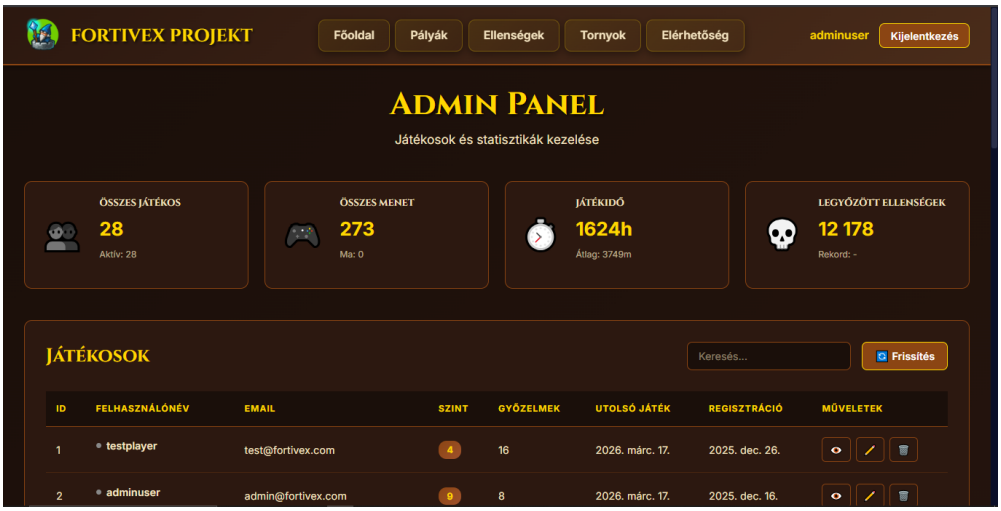
Az Admin Panel kizárólag az adminisztrátorok számára elérhető felület, amelyen átfogó képet kaphatnak a játék statisztikáiról és a regisztrált játékosokról. A panel tetején négy fontos összesített adat jelenik meg: az **összes regisztrált játékos** száma, az **összes lejátszott menet**, a játékosok által összesített **játékidő**, valamint a **legyőzött ellenségek** száma.

Az oldal alsó részén található a **játékosok listája**, amelyben az összes regisztrált felhasználó megtalálható. A lista tartalmazza a játékosok azonosítóját, felhasználónevét, email-címét, szintjét, győzelmeinek számát, utolsó játékának dátumát és regisztrációjának időpontját.

A keresőmező segítségével gyorsan megtalálható bármely játékos, a **Frissítés** gombra kattintva pedig az adatok azonnal aktualizálhatók.

Minden egyes játékos sorában három műveleti gomb található:

- **Szem gomb** – Erre kattintva az admin megtekintheti az adott játékos részletes statisztikáit, úgymint a győzelmek számát, az összes lejátszott játékot, a legyőzött ellenségek számát és az összegyűjtött arany mennyiségét.
- **Ceruza gomb** – Erre kattintva lehetőség nyílik a játékos **felhasználónevének** és **email-címének** módosítására. A változtatásokat a **Mentés** gombbal lehet véglegesíteni, vagy a **Mégse** gombbal visszavonni.
- **Kuka gomb** – Erre kattintva az admin törölheti az adott játékost a rendszerből. A törlés előtt egy megerősítő ablak jelenik meg, amely figyelmezteti az admint, hogy **a művelet nem vonható vissza.**



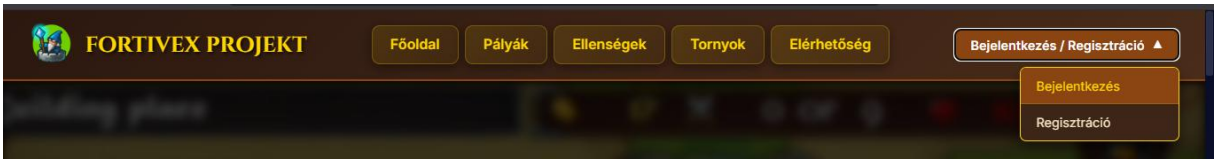
25. ábra: Admin nézet

6.4. Frontend: Weboldal, egy felhasználó szemével

A Fortivex Projekt weboldala egy letisztult, középkori hangulatú felületen keresztül mutatja be a játék világát. Az alábbiakban bemutatjuk, hogyan navigálhat egy felhasználó az oldalon – a főoldaltól kezdve egészen a regisztrációig és bejelentkezésig –, hogy minél egyszerűbben és gyorsabban megismerkedhessen a játék tartalmával.

Főoldal és navigáció

A weboldal tetején található a navigációs sáv, ahol a következő menüpontok érhetők el: **Főoldal, Pályák, Ellenségek, Tornyok** és **Elérhetőség**. A jobb felső sarokban található a **Bejelentkezés / Regisztráció** gomb – erre kattintva legördül egy menü, ahol kiválasztható a **Bejelentkezés** vagy a **Regisztráció** lehetőség.



26. ábra: Fejlécen a belépés/regisztráció

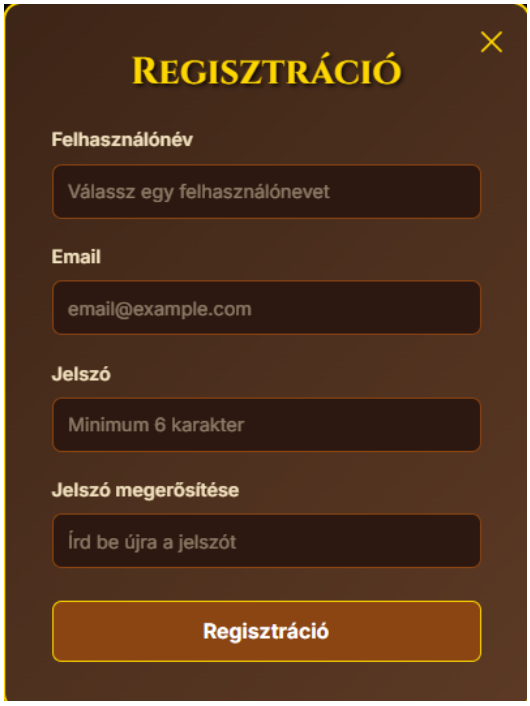
Bejelentkezés

A Bejelentkezés menüpontra kattintva egy felugró ablak jelenik meg. Itt meg kell adni a **felhasználónevet** és a **jelszót**. Lehetőség van a **„Maradj bejelentkezve"** opció bejelölésére is. A **Bejelentkezés** gombra kattintva lehet belépni a fiókba. Ha elfelejtetted a jelszavadat, kattints az **„Elfelejtett jelszó? Kattints ide!"** linkre.

27. ábra: Bejelentkezés panel

Regisztráció

Új fiók létrehozásához válaszd a **Regisztráció** lehetőséget. A megjelenő ablakban add meg a kívánt **felhasználónevet**, az **email címedet**, majd hozz létre egy **jelszót** (minimum 6 karakter), és erősítsd meg azt a **„Jelszó megerősítése"** mezőben. Végül kattints a **Regisztráció** gombra a fiók létrehozásához.



28. ábra: Regisztráció panel

Elfelejtett jelszó – Jelszó helyreállítása

Ha a felhasználó elfelejtette a jelszavát, a bejelentkezési ablak alján található **„Elfelejtett jelszó? Kattints ide!"** linkre kattintva indíthatja el a helyreállítási folyamatot.

A linkre kattintás után egy új oldal nyílik meg, ahol a felhasználónak meg kell adnia a fiókjához tartozó **email-címét**. Miután beírta az email-címet és elküldte a kérést, a rendszer egy **helyreállítási emailt** küld a megadott címre.

Az emailben található **hivatkozásra kattintva** a felhasználó egy biztonságos oldalra kerül, ahol lehetősége van **új jelszót beállítani** a fiókjához. Az új jelszó megadása után a felhasználó ismét be tud jelentkezni a fiókjába a megszokott módon.

Fontos: Ha nem érkezik meg az email néhány percen belül, érdemes ellenőrizni a **spam vagy levélszemét mappát** is, mivel egyes levelezőrendszerek oda sorolhatják az automatikusan küldött üzeneteket.

Profil oldal – Statisztikák

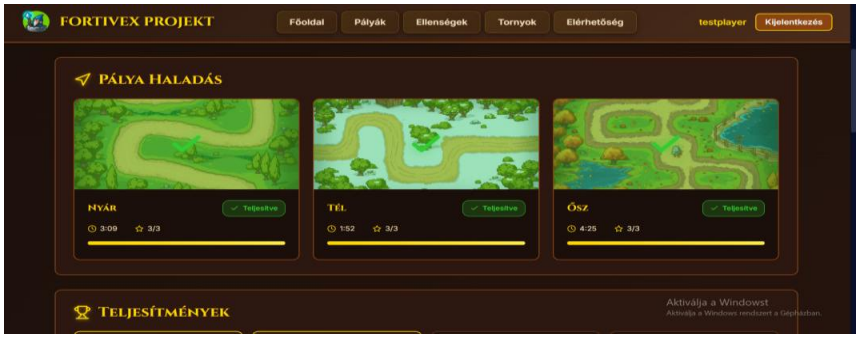
Bejelentkezés után a játékos a saját profil oldalára kerül, ahol személyre szabott üdvözléssel találkozik. Az oldal célja, hogy a játékos egy helyen követhesse nyomon fejlődését és eredményeit.

A profil oldalon négy fő statisztika jelenik meg. A **szint** mutatja a játékos aktuális fejlettségi szintjét, valamint az XP sávon jól látható, hogy mennyi tapasztalati pont szükséges a következő szint eléréséhez. A **győzelmek** száma megmutatja, hány csatát nyert meg a játékos az összes lejátszott menetből. Az **összegyűjtött arany** az eddig megszerzett arany teljes mennyiségét mutatja, illetve a jelenleg rendelkezésre álló összeget is. Végül a **legyőzött ellenségek** száma ad átfogó képet arról, hogy a játékos összesen hány ellenséget győzött le a játék során.

29. ábra: Belépés után a statisztikáinkat látjuk

Pályahaladás

A profil oldalon a **Pályahaladás** szekció áttekintést nyújt a játékos által teljesített pályákról. Minden egyes pályánál látható a teljesítés állapota, az elért idő, valamint a megszerzett csillagok száma. A képernyőn látható, hogy a játékos mindhárom elérhető pályát – **Nyár, Tél** és **Ősz** – sikeresen teljesítette, mindegyiken maximális, **3/3 csillagos** eredménnyel.



30. ábra: Pályahaladás

7.Telepítés és deployment folyamatok

Fortivex Projekt egy többplatformos (Windows, Linux, Android) Unity alapú 2D játék, amelyhez egy webes felület is tartozik a letöltések és információk biztosítására. A rendszer két fő részből áll:

- **Kliens (játék build)** – Unity-ben készített futtatható alkalmazás
- **Webes felület** – statikus weboldal, amelyről a játék letölthető
- **Backend API (opcionális)** – játékstatisztikák, bejelentkezés és adatok kezelésére

A telepítés és deployment folyamata ezek összehangolt kiadását jelenti.

7.1. Játék telepítése

A játék letölthető a hivatalos weboldalról az alábbi linken:

Lnk.ink/fortivex

A weboldalon a felhasználó ki választhatja a megfelelő platformot:

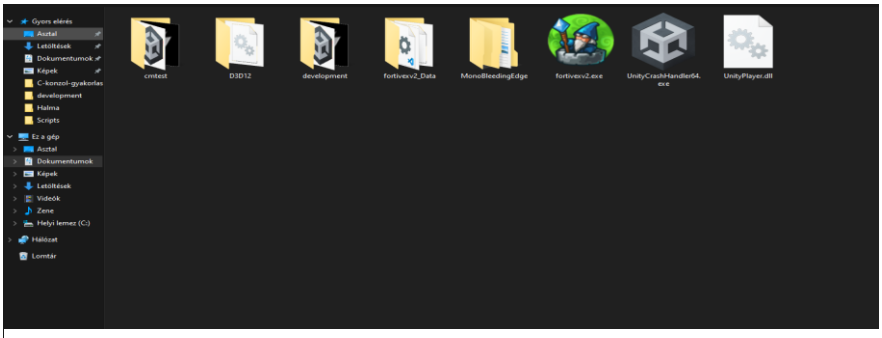
- Windows (.zip)
- Linux (.zip)
- Android (.apk)

7.2. Windows telepítés

A Windows verzió egy tömörített (.zip) fájlként érkezik, amely tartalmazza a teljes futtatókörnyezetet.

Lépések:

1. A `.zip` fájl letöltése után csomagold ki egy tetszőleges mappába



31. ábra: A játék a gépen

Futtatás:

Csak egyszerűen el kell indítani a `fortivexv2.exe`-t és élvezhetjük a játékot.

7.3. Linux telepítés

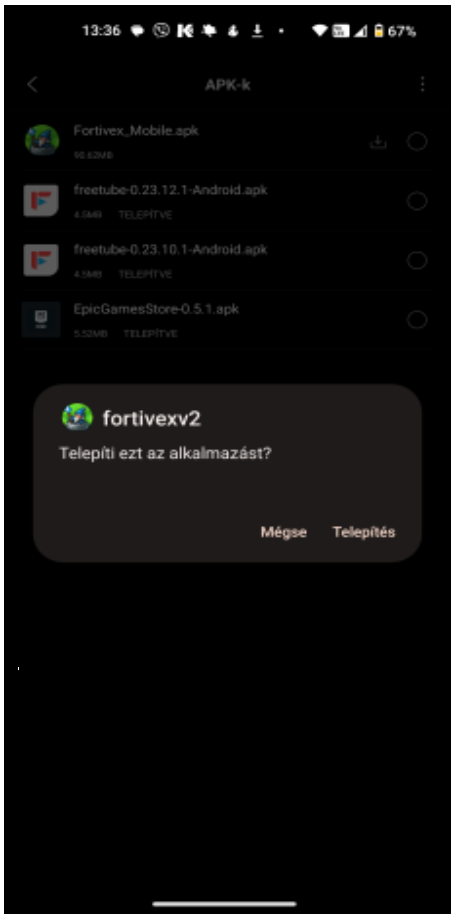
Mint Windows-nál, csak telepíteni kell, kicsomagolni és mehet is a játék.

7.4. Mobil telepítés

Töltsd le az `.apk` fájlt

- Engedélyezd az **Ismeretlen forrásokból származó alkalmazások telepítését.**
- Telepítsd az `APK-t`.
- Indítsd el az alkalmazást.

33. ábra: Engedélyezés a telepítésre



32. ábra: Telepítés

8. Munkamegosztás és projektmenedzsment

Egy szoftverfejlesztési projekt sikere nem csupán a technikai tudáson, hanem a csapaton belüli hatékony együttműködésen és a feladatok precíz delegálásán múlik. A Fortivex fejlesztése során az agilis módszertan elemeit ötvöztük a szigorú minőségbiztosítási elvárásokkal, biztosítva, hogy minden csapattag a saját kompetenciáinak legmegfelelőbb területen járuljon hozzá a közös célhoz.

A munkamegosztás során törekedtünk a felelősségi körök egyértelmű lehatárolására, miközben fenntartottuk a folyamatos tudásmegosztást a frontend, a backend és a játékfejlesztési divíziók között. Ebben a fejezetben bemutatjuk a csapat tagjainak egyéni hozzájárulását, a munkafolyamatok nyomon követésére használt eszközrendszerünket, valamint azt az időbeli ütemtervet, amely mentén a kezdeti koncepciótól eljutottunk a végleges, publikálásra a kész termékig.

8.1. Feladatkörök a csapaton belül

Medvecz Martin elsődleges feladata a játék központi motorjának, a `Unity` alapú asztali alkalmazásnak a fejlesztése volt. Felelősségi körébe tartozott a játékmechanikák (toronyépítés, ellenség-logika, hullámkezelés) implementálása, valamint a szoftver alapvető struktúrájának kialakítása. Emellett ő felelt a verziókezelésért (Git), biztosítva a forráskód integritását és a gördülékeny közös munkát. Adatbázis architektusként az `SQL` sémák tervezésével és a játékbeli adatok strukturált tárolásának kialakításával teremtette meg a projekt adatkezelési alapjait.

Vigh Gréta a projekt vizuális és felhasználói élményért felelős rétegeit alakította ki. Feladatai közé tartozott a reszponzív webes frontend fejlesztése, amely a játékosok számára biztosít hozzáférést a statisztikákhoz és a közösségi funkciókhoz. Kiemelt szerepet vállalt a mobilalkalmazás (Android) optimalizálásában, biztosítva, hogy a játékélmény érintőképernyős eszközökön is kifogástalan legyen. A fejlesztési ciklus végén a minőségbiztosításért is ő felelt: átfogó tesztelési folyamatokat hajtott végre a különböző platformokon a technikai hibák és a játékmenetbeli egyenetlenségek kiszűrése érdekében.

Halász-Szabó Dániel a rendszer „láthatatlan” motorjáért, a szerveroldali logikáért felelt. Backend fejlesztőként ő készítette el az **`ASP.NET Core`** alapú **`REST API`**-t, amely hidat képez a `Unity` kliens, a weboldal és az adatbázis között. Munkája során kiemelt figyelmet fordított a `deployment` folyamatokra, vagyis a rendszer élesítésére és a szerverkörnyezet konfigurálására, hogy az alkalmazás stabilan elérhető legyen. Emellett ő koordinálta a műszaki dokumentáció összeállítását, rögzítve a fejlesztés során alkalmazott megoldásokat és a rendszer felépítésének részleteit.

8.2. Munkafolyamat nyomon követése (Trello/GitHub)

A projekt átláthatósága és a határidők betartása érdekében modern projektmenedzsment eszközöket alkalmaztunk. A fejlesztés során a **Trello** platformján egy egyedi **Kanban táblát** hoztunk létre, amely lehetővé tette a feladatok valós idejű nyomon követését és a csapaton belüli szoros koordinációt.

A Trello használata során az alábbi szigorú módszertant követtük:

- **Egyéni és univerzális feladatok:** A táblán külön kezeltük a személyre szabott fejlesztési részfeladatokat és a teljes projektet érintő mérföldköveket.
- **Képi igazolás alapú lezárás:** Egy feladatot csak akkor tekintettünk befejezettnek és pipáltunk ki a rendszerben, ha a felelőse csatolt hozzá egy képes bizonyítékot (screenshotot vagy videót), amely igazolta a funkció hibátlan működését. Ez a belső ellenőrző rendszer garantálta, hogy csak ellenőrzött kód kerüljön tovább a következő fázisba.
- **Státuszkezelés:** A kártyákat a „folyamatban lévő", „tesztelésre váró" és „kész" oszlopok között mozgattuk, így bármelyik pillanatban látható volt a projekt aktuális készültségi foka.

A technikai megvalósítás és a forráskód kezelése a **GitHub** felületén zajlott. A verziókezelés során törekedtünk a beszédes commit üzenetekre és a logikus ágkezelésre, így a Trello-ban dokumentált feladatok közvetlenül visszakövethetőek voltak a kódbázis frissítéseiben is. Ez a kettős rendszer (Trello a feladatoknak, GitHub a kódnak) biztosította, hogy a Fortivex fejlesztése szervezett keretek között, dokumentált módon haladjon a végleges átadása felé. A projekt forráskódjának integritását és a fejlesztés technikai diverzitását a GitHub által generált statisztikák segítségével követtük nyomon. Ez az elemzés rávilágít arra, hogy a Fortivex nem csupán egyetlen technológiára épül, hanem egy komplex, többnyelvű ökoszisztéma.

Forráskód-összetétel (Linguist statisztika)

A projekt kódmennyiségének megoszlását egy kördiagram szemlélteti, amely hűen tükrözi a **Vue 3** és **Vite** alapú modern frontend architektúra dominanciáját. A statisztikák alapján látható, hogy a fejlesztés jelentős részét a **JavaScript (JS)** tette ki, ami a típusbiztonság és a skálázhatóság iránti elkötelezettségünket igazolja. A **Vue** fájlok magas aránya a komponensalapú fejlesztés eredménye, ahol a logika, a sablon és a stílus szoros egységet alkot. A **TypeScript (TS)** jelenléte a konfigurációs fájlokban és a Vite ökoszisztéma kiegészítőiben érhető tetten, míg a **JSON** állományok az adatstruktúrák, a nyelvi fájlok és a csomagkezelési beállítások (package.json) fontosságát jelzik. Ez az eloszlás bizonyítja, hogy a projekt során kiemelt figyelmet fordítottunk a modern webes szabványok betartására és a tiszta, jól tipizált kódbázis fenntartására.

A projekt technikai sokszínűségét tovább árnyalja a UNITY környezetben végzett munka, ahol a játékmenet alapjait fektettük le. Míg a webes felületen a VUE komponensek dominálnak, addig a játéklogika és az interakciók motorja a UNITY keretein belül kelt életre. A fejlesztés során a TS és a JS kódok közötti egyensúlyozás tette lehetővé, hogy a rendszer egyszerre legyen rugalmas és stabil. A JSON alapú konfigurációk és adatfájlok biztosították a zökkenőmentes átjárhatóságot a frontend és a backend rétegek között, garantálva az adatok egységességét. Ez a technológiai ökoszisztéma nem csupán a kódmennyiségről szól, hanem arról a precizitásról is, amellyel a különböző nyelveket és eszközöket egyetlen működő egésszé kovácsoltuk. Végső soron a VUE és a TS ilyen arányú jelenléte a statisztikákban a projekt modern és professzionális megközelítését támasztja alá. Az ezt követő oldalon látható diagram vizuálisan is szemlélteti ezt a belső arányt, pontosan megmutatva, mekkora szeletet tesz ki a teljes kódbázisból az említett technológiák és adatformátumok összessége.

Commit-aktivitás és fejlesztési dinamika

A fejlesztés intenzitását leginkább a `commitok` száma és azok időbeli eloszlása szemlélteti. A statisztikai adatok alapján látható, hogy a projekt életciklusában több kiugró aktivitási hullám is megfigyelhető volt, amelyek szorosan kapcsolódtak a korábban említett mérföldkövekhez. A magas `commit`-szám nem csupán a megírt kódsorok mennyiségét jelzi, hanem a folyamatos visszacsatolást és az agilis munkamenetet: a csapat tagjai apró, atomi frissítésekben rögzítették a változtatásokat, ami megkönnyítette a hibakeresést és a verziók közötti visszakövethetőséget. A grafikonon látható aktivitási csúcsok jellemzően a nagy „átalakítási" időszakokra, a `Unity`-mechanikák integrálására, valamint a záró tesztelési fázisra koncentrálódtak. Ez a dinamika igazolja a csapat kitartását és azt a folyamatos iteratív folyamatot, amely során a Fortivex egy kezdeti vázlatból stabil, `deploymentre` kész szoftverré érett.

8.3. Időbeosztás és mérföldkövek

A fejlesztés kezdeti szakaszában a legfontosabb feladatunk a közös vízió megteremtése és a technológiai alapkövek lefektetése volt. Ebben az időszakban a hangsúly a tervezésen és a későbbi hatékony munkát támogató rendszerek kiépítésén volt.

1. hét: Projektindítás és a kollaborációs terek kialakítása: A legelső lépésünk a strukturált munkavégzéshez szükséges keretek létrehozása volt. Regisztráltunk a Trello platformjára, ahol felállítottuk a projekt Kanban tábláját, meghatározva a kezdeti oszlopokat és munkafolyamatokat. Ezzel párhuzamosan megkezdődtek a kötetlen ötletelések a játék témájáról, ahol minden csapattag bedobta a saját elképzeléseit, legyen szó sci-fi, fantasy vagy történelmi környezetről.

2. hét: Koncepcióalkotás és a műfaj véglegesítése: A második héten az összegyűjtött ötleteket elkezdtük szűrni és rendszerezni. Hosszas mérlegelés után döntöttünk a Tower Defense műfaj mellett, mivel ez kínálta a legjobb egyensúlyt a komplex programozói feladatok (útkeresés, entitás-kezelés) és a látványos játékmenet között. Ebben a fázisban már konkrét vázlatokat készítettünk a Fortivex alapvető játékmechanikáiról, például a tornyok és az ellenséges hullámok kapcsolatáról.

3. hét: Vizuális irányvonal és a 2D grafika melletti döntés: A harmadik hét a játék esztétikai világának meghatározásáról szólt. A csapat egyhangúlag a 2D-s megjelenítés mellett tette le a voksát, mert ez tette lehetővé a legtisztább stratégiai átláthatóságot a játékos számára. Elkezdtünk kísérletezni a stílusokkal; olyan megjelenést kerestünk, amely egyedi hangulatot kölcsönöz a projektnek, és amelyhez a későbbiekben egységes asset-készletet tudunk gyártani vagy generálni.

4. hét: Technológiai stack és adatmodellezés: Ezen a ponton rögzítettük azokat a szoftveres eszközöket, amelyekre a teljes projekt épül. A játékmotor tekintetében a `Unity` mellett döntöttünk a kiváló multiplatform támogatása és a `C#` nyelv ismerete miatt. Meghatároztuk a backend (`ASP.NET Core`) és a verziókezelés (GitHub) paramétereit is. Ezzel egy időben elkészítettük az első SQL táblaterveket, meghatározva, hogyan fogjuk tárolni a felhasználói adatokat és a statisztikákat, valamint elindult a frontend fejlesztés is Vue3 + Vite alapokon, lefektetve a navigáció és a regisztrációs felület logikai alapjait.

5. hét: A Unity fejlesztés kezdete és a játéktér alapjai: Ezen a héten történt meg az áttörés a látványtervektől a működő prototípus felé. Elkezdtük a `Unity` környezetben a pálya fizikai alapjainak (grid-rendszer, útvonalak) kialakítását, és megjelentek az első ellenséges karakterek is. Ekkor készült el a wave-menedzsment logikájának első változata, amely már képes volt koordinálni az ellenfelek ütemezett érkezését és a hullámok közötti szüneteket.

6. hét: Egységek fejlesztése és a backend alapok: A hatodik héten a tornyok és a katonák béta fázisba kerültek, ami azt jelenti, hogy az alapvető harci interakciók már működtek. Ezzel párhuzamosan a háttérben az `ASP.NET Core API` alapfunkciói is formát öltöttek, lehetővé téve, hogy a játék és a szerver közötti kommunikáció első vonalai kiépüljenek. A webes frontend ezen a ponton már egy rendezett kezdőlappal rendelkezett.

7. hét: Felhasználói rendszerek, biztonsági szerepkörök és HUD: A hetedik hét a hitelesítés és a játékfelület jegyében telt. Finomhangoltuk a frontend bejelentkezési és regisztrációs folyamatait, ügyelve a felhasználói élményre. Az `API` oldalon bevezettük az adminisztrátori és normál felhasználói szerepköröket. A `Unityben` implementáltuk a `Head-Up Displayt (HUD)`, így a játékos már láthatta az életerejét, pénzét és a fejlesztési lehetőségeket. Elindítottuk a projekt-specifikus Wiki oldalt is, amely egyfajta belső enciklopédiaként szolgál a tornyok és ellenfelek tulajdonságainak megismeréséhez.

8. hét: Mérföldkő-ellenőrzés és mobil prototípus: A nyolcadik hetet a reflexióra szántuk. Egy átfogó bemutató keretében a csapat közösen vizsgálta felül az addigi eredményeket. Ez a belső audit kritikus fontosságú volt, mert itt szembesültünk azokkal a technikai adósságokkal és apróbb logikai hibákkal, amelyeket a lendületes fejlesztés során addig félretettünk. A multiplatform irányelvnek megfelelően elkészítettük a mobilalkalmazás első prototípusát, amely első körben a bejelentkezési felület tesztelésére fókuszált Android környezetben.

9. hét: Priorizálás, kódminőség és vizuális effektek: A fejlesztési szakasz egyik legfontosabb hetén nagyszabású „takarítást" végeztünk a kódbázisban. A feladatokat fontossági sorrendbe rendeztük, és megkezdtük a refaktorálást, azaz a kód szerkezeti javítását. Emellett a csapat a `Unityben` rejlő lehetőségek maximális kihasználására törekedett: a hangsúly a bonyolultabb torony-logikákon (pl. területre ható sebzés, lassítás) volt, és bevezettük a vizuális visszacsatolást javító részecske-effekteket (Particle Systems), amelyek a lövéseket, robbanásokat és mágikus hatásokat tették sokkal élvezhetőbbé.

10. hét: A nagy Frontend Makeover és játékegyensúly: Ez a hét vízválasztó volt a projekt arculatában. Úgy döntöttünk, hogy a korábbi webes designt teljesen elvetjük, és egy sokkal komolyabb, sötétebb tónusú, professzionálisabb megjelenést hozunk létre. Az új esztétika már tökéletesen harmonizált a játék misztikus világával. A megújult frontend finomhangolása mellett megkezdődött az intenzív játékmenet-tesztelés is: elsődleges célunk a mechanikák kiegyensúlyozása volt, hogy a hullámok nehézsége és a tornyok költsége igazságos, de kihívást jelentő görbét írjon le.

11. hét: Rendszerintegráció és kényelmi funkciók: A tizenegyedik héten párhuzamosan zajlott a munka minden fronton. Az utolsó funkcionális elemeket adtuk hozzá a rendszerhez, beleértve a játékmenetet színesítő hangbeállításokat és audio-effekteket. Ügyeltünk rá, hogy a webes felület és a játék motorja közötti adatcsere minden apró funkciónál zökkenőmentes legyen.

12. hét: Deployment előkészítése és Support oldal: A záró szakasz nyitányaként megkezdődtek az élesítés előkészületei. A statisztikai adatokat és a regisztrációs modulokat teljes körűen összekapcsoltuk az `API`-val, biztosítva a perzisztens adattárolást. Ebben az időszakban készült el a frontend support oldala is, amely a felhasználók későbbi segítését hivatott szolgálni.

13. hét: Early Release tesztelés és hibaelhárítás: A korai fázisú kiadások tesztelése során felszínre kerültek az első éles környezeti hibák. Kiemelt figyelmet fordítottunk a frontend belépési anomáliákra, mint például a kereszt-eredetű erőforrásmegosztási (`CORS`) problémákra és a váratlan `500-as szerverhibákra`, melyeket módszeresen javítottunk a stabil működés érdekében.

14. hét: Élesítés (Deployment) és szerverkonfiguráció: A tizennegyedik héten sor került a rendszer tényleges élesítésére. Konfiguráltuk a szerverkörnyezetet, optimalizáltuk az adatbázis-kapcsolatokat, és a Fortivex elérhetővé vált a célplatformokon. Ez a mérföldkő jelentette a fejlesztési szakasz technikai csúcspontját.

15. hét: Utolsó simítások és marketing anyagok: A projekt lezárásaként elvégeztük a dokumentáció végső formázását és a tartalomjegyzék frissítését. A játékon belül apró, de fontos vizuális javításoka eszközöltünk, mint például a legyőzött ellenfelek után maradó maradványok megjelenítése. Elkészült továbbá a projektet népszerűsítő előzetes videó is, amely a frontend kezdőlapján kapott helyet.

16. hét: Unit tesztek és a projekt véglegesítése: Az utolsó héten a minőségbiztosításé volt a főszerep. Átfogó egységteszteket (`Unit Test`) futtattunk a kritikus kódblokkokon, és elvégeztük az utolsó `bugfixeket`. A hét végére a Fortivex egy teljesen prezentálható, stabil és minden szakmai követelménynek megfelelő állapotba került.

9. Tesztelés

A Fortivex projekt tesztelése mind a `backend`, mind a frontend oldalán átfogóan megtörtént, különböző eszközök és módszerek kombinálásával.

9.1. Frontend tesztelés

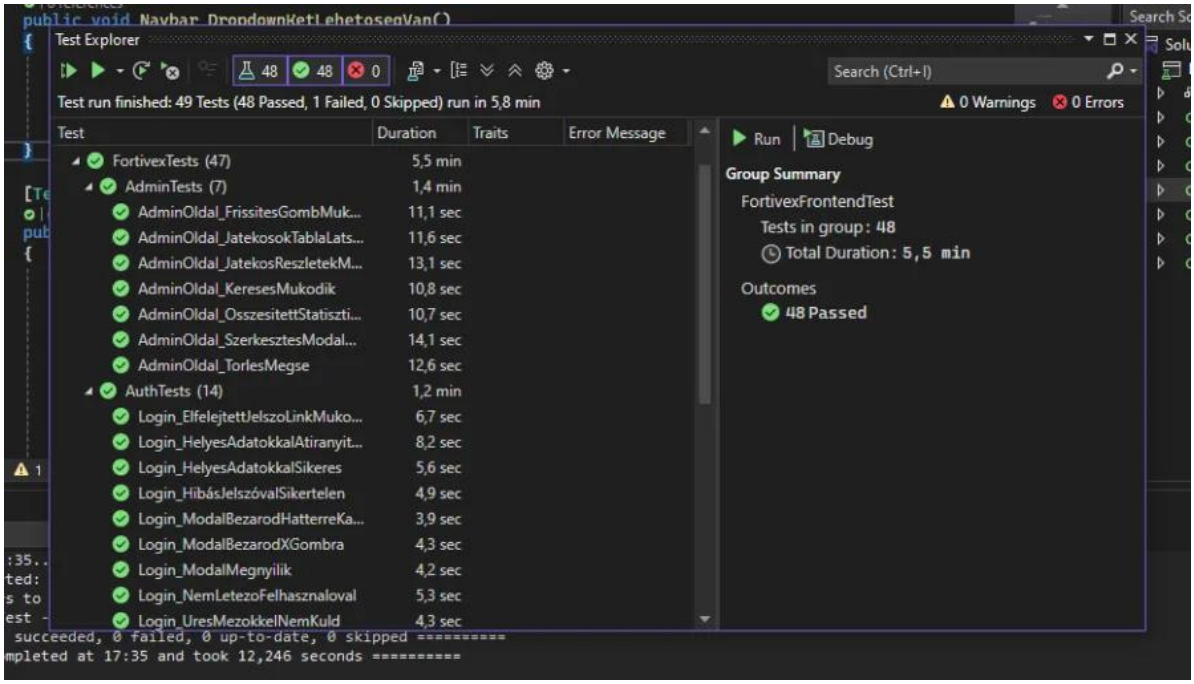
A frontend tesztelése két eszközzel történt. A **`Selenium`**-alapú automatizált UI tesztek `C#` nyelven, `NUnit` keretrendszerrel készültek, és a felhasználói folyamatokat ellenőrzik valós böngészőablakban – mint a bejelentkezés, regisztráció, navigáció és az admin felület működése. Összesen 48 tesztből 48 sikeres eredménnyel zárult, a teljes futásidő kb. 5,8 perc volt. A **`Google Lighthouse`** audit a Netlify-on élő alkalmazáson futott, ahol a teljesítmény 89, az akadálymentesség 93, a Best Practices 100, az SEO pedig 83 pontot ért el.

9.1.1. Selenium tesztek

A teszteket `C#` nyelven, `NUnit` keretrendszerrel írtuk, és `Visual Studio Test Explorerben` futtattuk. A projekt öt tesztosztályra tagolódik: `AdminTests`, `AuthTests`, `DownloadDropdownTests`, `NavbarTests` és `PlayerTests`, amelyeket a közös `TestBase.cs` konfigurál.

Összesen **48 teszt** fut le, amelyből **48 sikeres** – a teljes futásidő kb. 5,8 perc. A tesztek olyan felhasználói folyamatokat ellenőriznek automatikusan, mint a bejelentkezés, regisztráció, a navbar és a letöltési dropdown működése, valamint az admin felület és a játékos oldal elemei.

Minden teszt egy valós böngészőablakot nyit, elvégzi az adott műveletet (kattintás, kitöltés, navigáció), majd ellenőrzi az elvárt eredményt – így biztosítva, hogy a felhasználói felület valóban úgy működik, ahogy kell.



34. ábra: Sikeres Selenium tesztek

9.1.2. Lighthouse teszt

A Lighthouse mérést a Chrome DevTools segítségével végeztük a Netlify-on élő alkalmazáson. Az eredmények:

- **Teljesítmény:** 89
- **Akadálymentesség:** 93
- **Best Practices:** 100
- **SEO:** 83

35. ábra: Lighthouse tesztek

A 100-as Best Practices pontszám azt jelzi, hogy az alkalmazás modern webfejlesztési szabványokat követ. A teljesítmény és az SEO értékek még tovább javíthatók, de összességében az eredmények egy jól optimalizált, éles használatra kész webalkalmazást mutatnak.

9.2. Backend tesztelés

A backend tesztelése három keretrendszer segítségével valósult meg. Az `MSTest` és az `xUnit` egységteszteket biztosított az üzleti logika és az egyes komponensek helyes működésének ellenőrzésére. Az **`API tesztek`** a végpontok viselkedését vizsgálták – ellenőrizve, hogy a szerver a várt válaszokat adja-e vissza a különböző kérésekre, valamint, hogy a hibakezelés megfelelően működik-e.

9.2.1. xUnit teszt

Az `xUnit` egy `.NET`-es egységtesztelő keretrendszer, amellyel az alkalmazás egyes részeit **izoláltan** lehet tesztelni – tehát nem az egész rendszert egyszerre, hanem egy-egy konkrét metódust vagy osztályt. A Fortivex projektben ez elsősorban a **kontrollereket** és az **üzleti logikát** érinti.

Az `xUnit` egyik nagy előnye, hogy a tesztek **gyorsan futnak** és **nem igényelnek élő adatbázist vagy szervert** – a függőségeket általában mock objektumokkal helyettesítik, így a tesztek megbízhatóak és reprodukálhatók maradnak.

9.2.2. MSTest

Az `MSTest` a Microsoft hivatalos tesztkeretrendszere `.NET` alkalmazásokhoz, és a Fortivex projektben elsősorban a **hitelesítési folyamatok** ellenőrzésére lett alkalmazva.

Ez konkrétan azt jelenti, hogy a tesztek megvizsgálják, hogy a regisztráció és bejelentkezés mögötti logika helyesen működik-e – például érvénytelen jelszó esetén valóban hibát ad-e vissza a rendszer, hogy a token generálása és érvényesítése megfelelően zajlik-e, illetve hogy jogosulatlan kérések esetén az alkalmazás megfelelően megtagadja-e a hozzáférést.

Az `MSTest` az `xUnithoz` hasonlóan egységtesztelésre szolgál, azonban a Microsoft ökoszisztémával – például az **`ASP.NET Core-ral`** – különösen szorosan integrálható, ami megkönnyíti a hitelesítési réteg tesztelését. A keretrendszer **`[TestMethod]`** és **`[TestClass]`** attribútumai segítségével strukturáltan szervezhetők a tesztesetek, és a Visual Studio Test Explorerbe natívan illeszkedik, így az eredmények könnyen áttekinthetők.

9.2.3. API tesztek

Az API integrációs teszt abban különbözik az egységtesztektől, hogy nem csupán egy-egy izolált metódust vizsgál, hanem a **teljes kérés-válasz ciklust** ellenőrzi – az útvonaltól (routing) kezdve az üzleti logikán át egészen a visszaadott JSON válaszig. Ez azt jelenti, hogy a teszt úgy viselkedik, mintha egy valódi kliens – például egy mobilalkalmazás vagy böngésző – küldené a HTTP kéréseket a szerver felé, és ellenőrzi, hogy a rendszer minden rétege együttesen helyesen működik-e.

A tesztek a `.NET` beépített `WebApplicationFactory` megoldását használják, amely lehetővé teszi, hogy a teljes alkalmazást egy valódi szerver elindítása nélkül, memóriában futtassuk. Ez gyors, megbízható és környezetfüggetlen tesztelést tesz lehetővé – nincs szükség külső adatbázis-kapcsolatra vagy élő szerverre.

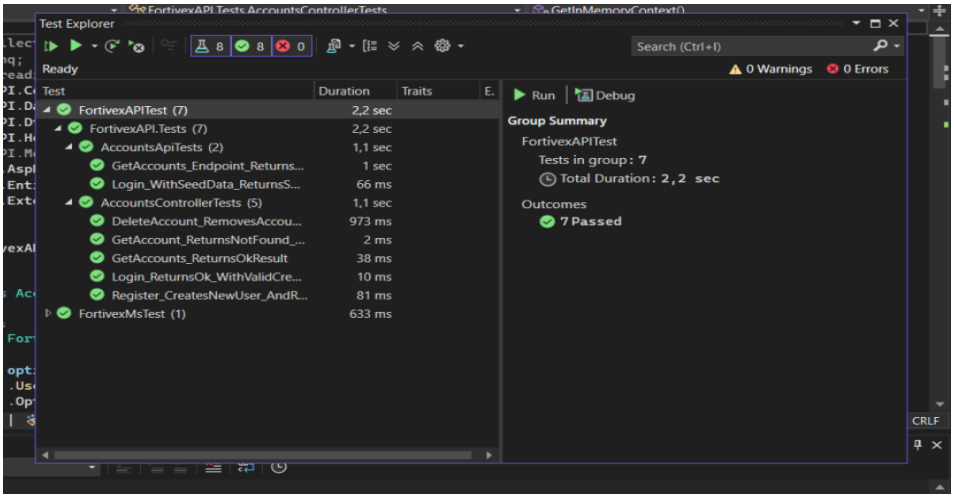
9.2.3.1 Tesztelt végpontok

A tesztek az `AccountsController` kritikus végpontjainak helyes működését igazolják:

- **Fiókok listázása:** Ellenőrzi, hogy az összes regisztrált felhasználó sikeresen lekérdezhető-e, és a visszaadott lista formátuma megfelel-e az elvárásoknak.
- **Egyedi lekérdezés:** Azonosító alapján keres meg egy adott fiókot. A teszt mind a létező, mind a nem létező ID esetét lefedi – utóbbinál a megfelelő hibaválasz visszaadását ellenőrzi.
- **Regisztráció:** Új felhasználók létrehozását teszteli, beleértve a szerepkörök (User/Admin) helyes kezelését és az adatbázisba való mentés sikerességét.
- **Bejelentkezés:** Ellenőrzi, hogy helyes felhasználónév és jelszó megadásakor a rendszer érvényes `JWT tokent` generál-e, illetve hibás adatok esetén megfelelő hibaüzenetet ad-e vissza.
- **Törlés:** Felhasználói fiókok eltávolítását vizsgálja, és azt, hogy a törlés után az adott fiók valóban nem elérhető-e többé a rendszerből.

Ezek a tesztfolyamatok biztosítják, hogy a backend-rendszer minden kritikus üzleti logikája stabilan és biztonságosan működjön, megakadályozva az illetéktelen hozzáféréseket vagy az adatok sérülését. Az automatizált ellenőrzéseknek köszönhetően a legkisebb kódmódosítás után is azonnal látható, ha egy végpont nem a várt módon reagál, így a fejlesztés során végig fenntartható maradt a rendszer integritása. Az alábbi képen látható zöld jelzések és a sikeres

futási eredmények pontosan ezt a megbízhatóságot és a végpontok hibátlan válaszadási készségét támasztják alá.



36. ábra: Sikeres API tesztek

Eredmények:

Az összes, azaz mind a **7 teszt sikeresen lefutott**, egyetlen hiba sem fordult elő. A tesztek a kritikus backend üzleti logikát teljes körűen lefedik, biztosítva, hogy az autentikációs és fiókkezelési funkciók az elvárásoknak megfelelően működnek éles környezetben is.

9.3. Tesztelések összegzése

A projekt tesztelési lefedettsége mind backend, mind frontend szinten kiterjedt: az egységtesztektől az API-ellenőrzéseken át a végfelhasználói felület automatizált vizsgálatáig. Az eredmények összességében egy stabil, jól működő alkalmazást mutatnak.

10. Összegzés és jövőbeli fejlesztési irányok

A Fortivex projekt egy teljes értékű, multiplatformos Tower Defense játékká nőtte ki magát, amely egyszerre teljesíti a szakmai és a játékélménybeli követelményeket. A fejlesztés során a csapat valós körülmények között sajátította el a modern szoftverfejlesztés eszköztárát: a `Unity` alapú játékmotortól az `ASP.NET Core` backenden át a `Vue 3`-ban készült webes felületig egy összetett, több platformot kiszolgáló rendszert hoztunk létre. A projekt nem csupán egy játék elkészítéséről szólt – a folyamat során a csapattagok megtanulták, hogyan kell egy valódi szoftverfejlesztési projektet a tervezéstől az élesítésig végigvinni, beleértve a verziókezelést, a projektmenedzsmentet és a minőségbiztosítást.

A tesztelési folyamatok – `Selenium`, `Lighthouse`, `MSTest`, `xUnit` és `API tesztek` – igazolják, hogy az alkalmazás stabil, biztonságos és éles használatra kész. A 48 sikeres frontend teszt, a 7 hibátlanul lefutó `API` teszteset és a Lighthouse 100 pontos Best Practices eredménye mind azt támasztja alá, hogy a fejlesztés során következetesen szem előtt tartottuk a minőségi szempontokat. A backend robusztus architektúrája és a JWT alapú hitelesítési rendszer pedig biztonságos alapot nyújt a jövőbeli bővítésekhez.

A jövőbeni fejlesztési irányok között elsősorban a tartalom bővítése szerepel: új pályák, ellenségtípusok és tornyok hozzáadásával gazdagítható a játékmenet, miközben a nehézségi görbe is tovább finomítható. Többjátékos funkció elkészítésében is gondolkodunk. Közösségi funkciók, mint klánrendszer vagy barátlista, szintén beilleszthetők a már meglévő API-infrastruktúrába. A webes felület teljesítmény- és SEO-pontszámainak javítása szintén reális cél a közeljövőben, ahogyan a WebGL-es böngészős verzió fejlesztése és az iOS platform támogatása is újabb felhasználói rétegeket nyithatna meg a projekt számára. A Fortivex így nem csupán egy lezárt iskolai projekt, hanem egy szilárd alapokra épített rendszer, amely készen áll a további fejlődésre.

11. Felhasznált források

Játék keretrendszer: Unity, Unity asset store

Weblap keretrendszer: Vue 3, Vite, Node.js

Programozás: Visual Studio 2022, Visual Studio Code

Képek és kódolási problémák: ChatGPT.com, Gemini.com, Claude.AI , w3schools.com, Unity asset store.

Ábrajegyzék

1. ábra: Iteratív és Agilis fejlesztési modell 7
2. ábra: Goblin dárdás és harcos játékbeli modellje 9
3. ábra: Ork játékbeli modellje 10
4. ábra: Sámán és a Voodoo játékbeli modelljei 11
5. ábra: A farkas játékbeli modellje 11
6. ábra: Denevér játékbeli modellje 12
7. ábra: A Monstrum tank játékbeli modellje 12
8. ábra: A gyalogos játékbeli modelljei és fejlődései 14
9. ábra: Az íjásztorony játékbeli modelljei és fejlődései 15
10. ábra: A mágustorony játékbeli modelljei és fejlődései 15
11. ábra: A bombázó torony játékbeli modelljei és fejlődései 16
12. ábra: A dárdavető torony játékbeli modelljei és fejlődései 17
13. ábra: A Csillaghullás játékbeli ikonja 18
14. ábra: A Meteorraj játékbeli ikonja 18
15. ábra: Belépés/Regisztráció 19
16. ábra: Start menü 19
17. ábra: Pályakiválasztási menü 19
18. ábra: Az első pálya kinézete 20
19. ábra: Toronyvásárlás 21
20. ábra: Közelharci torony képessége 24
21. ábra: A 9.ik kör végén a tank megjelenik 26
22. ábra: Component diagram 30
23. ábra: UML Diagram 32
24. ábra: A játék főoldala 33
25. ábra: Admin nézet 34
26. ábra: Fejlécen a belépés/regisztráció 35
27. ábra: Bejelentkezés panel 35
28. ábra: Regisztráció panel 36
29. ábra: Belépés után a statisztikáinkat látjuk 37
30. ábra: Pályahaladás 37
31. ábra: A játék a gépen 38
32. ábra: Telepítés 39

33. ábra: Engedélyezés a telepítésre 39
34. ábra: Sikeres Selenium tesztek 49
35. ábra: Lighthouse tesztek 49
36. ábra: Sikeres API tesztek 52